LAPORAN PRAKTIKUM DEVSECOPS

ENTERPRISE OBSERVABILITY STACK DI DOCKER: CENTRALIZED LOGGING FLUENT BIT SISTEM PMB DAN RESOURCE MONITORING PROMETHEUS GRAFANA

(PRAKTIKUM BAB 6 DAN BAB 7)



Disusun Oleh:

Muhamad Reza Muktasib
3126640060
D4 RPL IT B

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada arsitektur modern berbasis kontainer dan *microservices*, beban kerja berjalan secara dinamis, terdistribusi, dan bersifat fana (*ephemeral*). Kontainer aplikasi dapat dibuat, diskalakan, diperbarui, atau dimatikan sewaktu-waktu. Karakteristik ini menimbulkan tantangan besar terhadap visibilitas operasional dan keamanan: apabila log aplikasi dan metrik penggunaan sumber daya hanya tersimpan secara lokal di dalam *writable layer* masing-masing kontainer, maka saat terjadi kegagalan atau kontainer dihentikan, seluruh bukti kronologis kejadian akan musnah. Kondisi ini membuat proses pelacakan insiden (*incident response*), penelusuran akar masalah (*root cause analysis*), serta pemantauan kapasitas (*capacity planning*) menjadi sangat sulit dilakukan.

Oleh karena itu, sistem operasional perangkat lunak modern menuntut penerapan *Enterprise Observability Stack* yang komprehensif, mencakup dua pilar telemetri utama, yaitu **Logging** (*Bab 6*) dan **Monitoring/Metrics** (*Bab 7*). Pada Bab 6, diimplementasikan studi kasus nyata berskala multi-tier: **PMB DevSecOps Laboratory** (Sistem Penerimaan Mahasiswa Baru). Sistem ini mengorkestrasikan tiga layanan aplikasi (`registration-api`, `auth-service`, dan `verification-worker`) dengan pemisahan mutlak antara basis data transactional (`app-db`) dan basis data log terpusat (`log-db`). Aliran log ditangkap langsung dari `stdout` menggunakan Docker `fluentd` logging driver, dibuffer di media fisik (*disk buffer*) oleh Fluent Bit v4.2 untuk mencegah *data loss* akibat *backpressure*, lalu disimpan ke dalam tabel PostgreSQL berformat biner `JSONB` yang diindeks GIN.

Melanjutkan fondasi logging tersebut, Bab 7 mengintegrasikan infrastruktur *Resource Monitoring* menggunakan Prometheus sebagai mesin pengumpul (*scraper*) dan basis data deret waktu (*Time-Series Database* / TSDB). Prometheus menarik metrik dari Node Exporter untuk host fisik/VM dan cAdvisor untuk konsumsi CPU, memori, serta I/O cgroup seluruh kontainer Docker (termasuk kontainer sistem PMB). Data deret waktu tersebut diolah dengan query PromQL, distabilkan melalui *recording rules*, diuji keandalannya menggunakan *alerting rules*, dan divisualisasikan dalam *dashboard* Grafana berbasis *JSON-as-Code*. Sinergi kedua pilar telemetri ini disatukan secara nyata pada panel Grafana melalui integrasi *cross-network* ke `log-db` PMB menggunakan akun *read-only*, menghasilkan satu kesatuan visibilitas terpadu yang andal, aman, dan berstandar industri.

1.2 Tujuan Praktikum

1. Mampu merancang dan mengoperasikan arsitektur *Centralized Logging* pada sistem multi-tier PMB menggunakan Docker `fluentd` logging driver dan Fluent Bit.
2. Mampu menerapkan pemisahan arsitektural antara basis data transactional (`app-db`) dan basis data telemetri terpusat (`log-db`) berkolom `JSONB` dengan indeks GIN.
3. Mampu menerapkan mekanisme pelacakan terdistribusi (*distributed tracing*) lintas layanan menggunakan korelasi identifier unik `request_id`.
4. Mampu merekonstruksi kronologi kegagalan sistem secara deterministik menggunakan query SQL analitik pada saat simulasi *dependency timeout*.
5. Mampu membuktikan ketahanan antrean (*buffering resilience*) Fluent Bit saat backend basis data log mengalami *outage* sementara (*zero log loss*).
6. Mampu menegakkan prinsip keamanan *Data Minimization* dan sanitasi PII agar kredensial atau informasi pribadi tidak bocor ke repositori log.
7. Mampu menganalisis komparasi arsitektural dan fungsional antara Prometheus (*Metrics Engine*) dan Fluent Bit (*Log Shipper*) dalam ekosistem *observability*.
8. Mampu membangun *monitoring stack* terisolasi berbasis Prometheus, cAdvisor, Node Exporter, dan Grafana menggunakan Docker Compose.
9. Mampu menyusun query PromQL, mendefinisikan *recording rules*, serta merancang *alerting rules* yang divalidasi keabsahannya menggunakan `promtool`.
10. Mampu mengintegrasikan *datasource* metrik Prometheus dan *datasource* log PostgreSQL PMB ke dalam satu dashboard Grafana terpadu secara aman.

BAB II
DASAR TEORI

2.1 Arsitektur Multi-Service PMB dan Docker Fluentd Logging Driver

Dalam metodologi *Twelve-Factor App*, aplikasi cloud-native tidak boleh mengelola penulisan berkas log lokal ke disk. Setiap proses aplikasi wajib meng-emit seluruh log record ke aliran standar `stdout` dan `stderr`. Docker Engine bertindak sebagai penangkap aliran tersebut melalui mekanisme *logging driver*. Pada laboratorium PMB ini, driver dikonfigurasi ke mode `fluentd` dengan konfigurasi deklaratif berikut.

Parameter Konfigurasi	Nilai Konfigurasi	Fungsi dan Penjelasan Teknis
`logging.driver`	`fluentd`	Menginstruksikan Docker daemon untuk mengirim log via protokol Forward binary
`fluentd-address`	`127.0.0.1:24224`	Alamat port host tempat engine Fluent Bit mendengarkan input forward
`fluentd-async`	`"true"`	Mode non-blocking agar kontainer aplikasi tidak tertahan saat collector lambat
`fluentd-buffer-limit`	`"4096"`	Batas kapasitas ring buffer memori internal Docker daemon untuk tiap kontainer
`tag`	`pmb.{{.Name}}`	Pola label routing dinamis yang menyertakan nama kontainer sumber

Penerapan opsi `fluentd-async: "true"` merupakan kontrol ketersediaan (*availability control*) yang sangat krusial. Jika mode *asynchronous* tidak diaktifkan (mode default *blocking*), ketika Fluent Bit mengalami lonjakan beban atau berhenti sementara, proses penulisan ke `stdout` oleh aplikasi Flask atau Gunicorn akan tertahan (*hung*), yang pada akhirnya melumpuhkan seluruh portal pendaftaran mahasiswa.

2.2 Topologi Dual Database: Transactional (app-db) vs Logging (log-db)

Sistem PMB menegakkan pemisahan mutlak (*separation of concerns*) melalui topologi dua basis data PostgreSQL independen berikut.

Parameter Pembeda	Application Database (`app-db`)	Centralized Log Database (`log-db`)
Tujuan Utama	Menyimpan status transaction bisnis pendaftaran PMB	Menyimpan jejak audit telemetri kejadian sistem
Model Operasi Data	*Read-heavy* dan *Update-intensive* (ACID Transactions)	*Append-only* ingest berkecepatan tinggi
Skema Tabel Utama	Tabel `registrations` (kolom relasional terstruktur kaku)	Tabel `logs.fluentbit` (kolom `tag`, `time`, `data JSONB`)
Karakteristik Indeks	B-Tree pada `user_ref`, `status`, dan `request_id`	B-Tree pada `time DESC` dan GIN pada kolom `data`
Dampak Kegagalan	Jika down, aplikasi tidak dapat memproses pendaftaran	Jika down, transaction tetap jalan karena log dibuffer
Kebijakan Akses	Diberikan kepada akun aplikasi PMB (`app_user`)	Diberikan kepada akun log shipper (`fb_user`)

Penyimpanan log pada `log-db` memanfaatkan tipe data `JSONB` (*Binary JSON*). Format biner terurai ini memungkinkan PostgreSQL mengekstrak atribut bersarang (seperti `data ->> 'service'`, `data ->> 'severity'`, atau `(data ->> 'log')::jsonb ->> 'error_code'`) dan memanfaatkan indeks GIN (*Generalized Inverted Index*) untuk mengeksekusi query analitik secara instan tanpa melakukan pemindaian menyeluruh (*full-table scan*).

2.3 Mekanisme Distributed Tracing berbasis Correlation ID (request_id)

Setiap kali permintaan masuk ke portal PMB, sistem menghasilkan token unik berformat `REQ-PMB-<timestamp>-<random>` (misalnya `REQ-PMB-20261027-144310-9421`). Token ini disuntikkan ke dalam respon HTTP melalui header `X-Request-ID`, dicatat pada kolom tabel transaction di `app-db`, serta disematkan ke dalam setiap baris log records terstruktur yang di-emit oleh seluruh layanan (`registration-api`, `auth-service`, `verification-worker`). Dengan demikian, teknisi operasi dapat merekonstruksi transaction lifecycle secara utuh (*end-to-end tracing*) hanya dengan memfilter satu nilai `request_id`.

2.4 Keamanan Data: Data Minimization dan PII Sanitization

Prinsip keamanan *Data Minimization* (GDPR dan UU PDP) mewajibkan sistem untuk mem-filter dan men-sanitize informasi sensitif sejak dari hulu. Pada modul aplikasi PMB (`app/common.py`), diterapkan sanitasi otomatis: kata sandi dan token sesi di-filter dan di-sanitize sehingga tidak pernah dicatat ke log, sedangkan Nama Lengkap dan Email mahasiswa hanya disimpan di `app-db` yang terenkripsi dan sengaja tidak di-emit ke aliran log Fluent Bit. Log hanya mencatat identifier anonim (`registration_id`, `request_id`), kode status pendaftaran, durasi eksekusi, serta kode kesalahan teknis (`error_code`).

2.5 Arsitektur Resource Monitoring: Prometheus Scraper, TSDB, Exporters, dan Grafana

Monitoring menyediakan pengukuran kuantitatif terhadap performa dan kesehatan infrastruktur melalui deret waktu (*time-series*). Prometheus menggunakan arsitektur berbasis *pull model*, di mana server Prometheus secara aktif menarik metrik dari endpoint HTTP `/metrics` milik target *exporter* pada interval waktu yang telah ditentukan (*scrape interval*).

Komponen Stack	Port Default	Peran Arsitektural dalam Monitoring
Prometheus	`9090`	Menarik metrik, menyimpan dalam TSDB lokal, mengevaluasi PromQL dan alerts
Node Exporter	`9100`	Mengekspos metrik level host Linux (CPU, memori, disk I/O, network)
cAdvisor	`8080` / internal	Menganalisis dan mengekspos metrik konsumsi kontainer Docker via cgroups
Grafana	`3000`	Memvisualisasikan data deret waktu dari berbagai *datasource* dalam dashboard

Setiap titik data deret waktu dalam Prometheus direpresentasikan oleh format umum berikut.

$$\text{metric_name}\{\text{label_name} = \text{"label_value"}\} \quad \text{value} \quad [\text{timestamp}]$$

Terdapat empat tipe metrik dasar dalam Prometheus.

1. **Counter** nilai kumulatif yang hanya bertambah atau di-*reset* ke nol saat proses *restart* (misalnya `node_cpu_seconds_total`). Analisis counter wajib menggunakan fungsi laju perubahan seperti `rate()` atau `increase()`.
2. **Gauge** nilai yang dapat naik dan turun sewaktu-waktu mencerminkan kondisi sesaat (misalnya `node_memory_MemAvailable_bytes` atau `container_memory_usage_bytes`).
3. **Histogram** mengukur frekuensi observasi dalam rentang ukuran tertentu (*buckets*) untuk perhitungan latensi atau durasi.
4. **Summary** menghitung kuantil (*quantiles*) secara langsung pada sisi *client*.

2.6 Komparasi Arsitektural: Prometheus vs Fluent Bit dalam Ekosistem Observability

Sering kali terdapat kesalahpahaman mengenai peran Prometheus dan Fluent Bit. Secara intuitif, banyak yang memandang bahwa *Prometheus adalah pengolah data penerima sedangkan Fluent Bit adalah logger-nya*. Pemahaman ini memiliki dasar yang tepat pada tataran praktis, namun secara arsitektur sistem terdistribusi, keduanya memiliki perbedaan fundamental pada tipe data telemetri, model aliran data (*data flow*), kapasitas penyimpanan, dan tujuan operasionalnya.

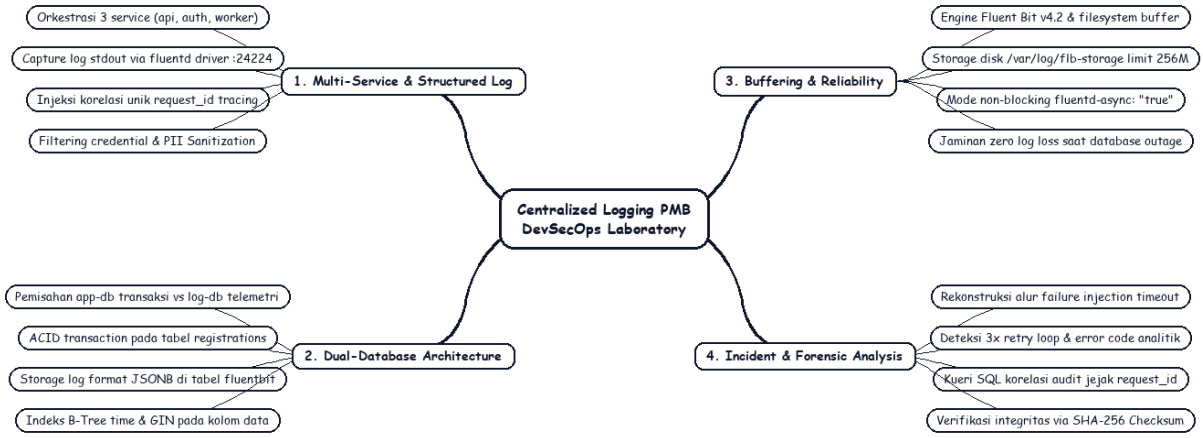
Prometheus dirancang secara spesifik sebagai **Metrics Engine** (pengumpul, penyimpan, dan pemroses data deret waktu numerik). Prometheus menjawab pertanyaan: *"Apa anomali yang sedang terjadi pada sistem saat ini?"* (misalnya: latensi API meningkat melampaui SLO, utilisasi CPU mencapai 95%, atau jumlah kontainer aktif turun drastis). Sebaliknya, Fluent Bit dirancang sebagai **Log Processor & Forwarder** (telemetry shipper untuk event records teks diskret dan JSON berstruktur). Fluent Bit menjawab pertanyaan: *"Mengapa hal tersebut dapat terjadi?"* (misalnya: menemukan berkas log yang mencatat *NullPointerException*, error koneksi basis data *timeout*, atau jejak serangan *SQL injection*).

Parameter Komparasi	Prometheus (Metrics Engine)	Fluent Bit (Log Processor / Forwarder)
Tipe Data Telemetri	Deret waktu numerik (*numerical time-series*) teragregasi	Kejadian teks diskret (*event logs*) dan objek JSON terstruktur
Model Aliran Data	**Pull Model** (Prometheus aktif melakukan HTTP *scrape* berkala)	**Push / Stream Model** (Aplikasi/Docker mengirimkan log secara terus-menerus)
Format data records	Nama metrik + sekumpulan label dimensi + nilai float64	Timestamp presisi + Tag routing + Payload teks / atribut JSON berstruktur
Karakteristik Storage	TSDB internal (*chunk compression*, rentang waktu retensi lokal)	Buffer memori/disk sementara, wajib di-*forward* ke storage eksternal (DB)
Kapasitas Cardinality	Rentan terhadap *high cardinality* (label bernilai unik besar)	Sangat toleran terhadap data berdimensi tinggi (ID transaction, IP, pesan teks)
Bahasa Query Data	**PromQL** (perhitungan `rate`, `sum`, `histogram_quantile`, derivasi)	**SQL / GIN Indexing / Lucene / LogQL** pada backend tujuan akhir
Beban Sumber Daya	Memerlukan RAM dan CPU signifikan untuk kalkulasi indeks TSDB	Sangat ringan (*lightweight*, berbasis bahasa C, konsumsi memori rendah)
Mekanisme Alerting	Evaluasi aturan matematis real-time (*threshold rules* & durasi `for`)	Berbasis filter pola teks atau didelegasikan ke SIEM / Elastic / OpenSearch
Kapan Digunakan	Memantau kesehatan sistem, kapasitas, SLI/SLO, dan deteksi dini	*Debugging*, audit keamanan, kepatuhan (*compliance*), analisis jejak error
Peran di DevSecOps	Detektif gejala: memberi peringatan saat terjadi anomali beban	Forensik akar masalah: menyediakan bukti kronologis insiden sistem

Berdasarkan komparasi di atas, Prometheus dan Fluent Bit bukanlah dua teknologi yang saling menggantikan (*mutually exclusive*), melainkan dua pilar yang saling melengkapi (*complementary*). Dalam alur penanganan insiden (*Incident Response Lifecycle*), alarm dari Prometheus memicu respon tim operasi, sedangkan log yang dikumpulkan Fluent Bit memberikan rincian untuk menemukan dan memperbaiki kerusakan kode atau konfigurasi secara presisi.

2.7 Peta Konsep: Centralized Logging PMB DevSecOps Laboratory

Alur kerja pengumpulan log dari kontainer multi-service PMB hingga penyimpanan persisten dan analisis diilustrasikan pada diagram *Mind Map* berikut.

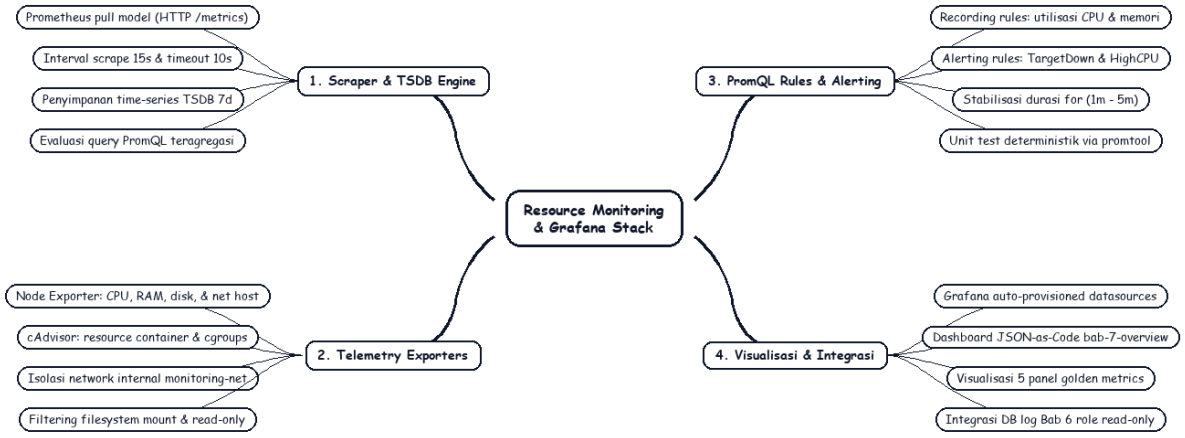


Berdasarkan pemetaan konsep di atas, arsitektur *Centralized Logging* PMB terbagi menjadi empat pilar berikut.

1. **Multi-Service dan Structured Log** mengumpulkan aliran `stdout` dan `stderr` kontainer menggunakan driver `fluentd` pada port host `24224`, melampirkan label tag dinamis `pmb.{{.Name}}`, serta mem-filter dan men-sanitize data PII.
2. **Dual-Database Architecture** memisahkan secara fisik basis data transactional ACID `app-db` dari basis data telemetri `log-db` berformat `JSONB` yang dilengkapi indeks inversi tergeneralisasi GIN.
3. **Buffering dan Reliability** menyediakan jaminan keutuhan data (*zero log loss*) melalui alokasi buffer penyimpanan fisik di disk `/var/log/flb-storage` bertipe `filesystem` dan batas kapasitas *backpressure* 256MB.
4. **Incident Investigation dan Forensic** memfasilitasi rekonstruksi kronologi transaction secara deterministik menggunakan query SQL analitik pada saat simulasi *dependency timeout*, audit keamanan kebocoran kredensial, serta pembuktian integritas artefak via SHA-256.

2.8 Peta Konsep: Resource Monitoring Prometheus dan Grafana

Arsitektur pemantauan performa sumber daya berbasis metrik dipetakan pada diagram *Mind Map* berikut.

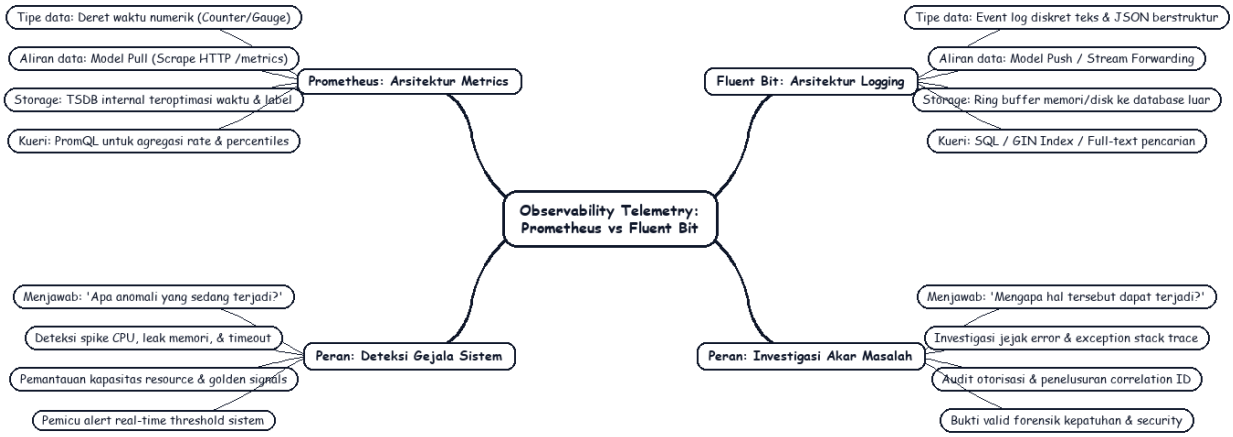


Berdasarkan pemetaan konsep di atas, arsitektur *Resource Monitoring* mencakup empat komponen utama berikut.

1. **Scraper dan TSDB Engine** mengatur Prometheus untuk menarik data metrik dari target setiap 15 detik dengan batas *timeout* 10 detik, menyimpannya ke dalam media penyimpanan *time-series* lokal dengan retensi 7 hari, dan mengevaluasi ekspresi PromQL secara teragregasi.
2. **Telemetry Exporters** menampilkan data mentah sistem tanpa mengekspos port ke publik luar. Node Exporter membaca kondisi CPU, memori, partisi disk, dan antarmuka jaringan host, sedangkan cAdvisor mengekstrak konsumsi sumber daya dan batasan cgroup kontainer secara real-time.
3. **PromQL Rules dan Alerting** memformulasikan metrik turunan melalui *recording rules* (seperti rasio utilisasi CPU dan memori) untuk mempercepat query *dashboard*, serta menyusun *alerting rules* seperti `TargetDown` dan `ContainerHighCPU` yang divalidasi keabsahannya menggunakan pengujian deterministik `promtool`.
4. **Visualisasi dan Integrasi** mendistribusikan *dashboard* Grafana terprovisi otomatis (*JSON-as-Code*) untuk menampilkan 5 panel metrik utama, serta membuka saluran komunikasi privat ke basis data `log-db` PMB menggunakan akun *read-only* untuk analisis korelasi silang (*cross-telemetry analysis*).

2.9 Peta Konsep: Sinergi Telemetri Prometheus vs Fluent Bit

Hubungan sinergis antara pemantauan metrik dan pengumpulan log dalam arsitektur DevSecOps dipetakan pada diagram *Mind Map* berikut.



Berdasarkan pemetaan komparatif di atas, keselarasan operasional antara Prometheus dan Fluent Bit diwujudkan melalui tiga aspek berikut.

1. **Prometheus sebagai Sensor Gejala (Metrics)** berfokus pada efisiensi pemrosesan deret waktu numerik berdimensi terkontrol. Prometheus mendeteksi secara dini kapan terjadi lonjakan beban, kepenuhan media penyimpanan, atau kegagalan target layanan melalui visualisasi tren dan pemicu alarm otomatis.
2. **Fluent Bit sebagai Instrumen Investigasi (Logs)** menangani tingginya keberagaman informasi (*high cardinality*) dari event records tekstual. Fluent Bit mengamankan jejak audit, pesan kesalahan aplikasi, dan riwayat otentikasi sehingga teknisi dapat mengetahui penyebab pasti mengapa gejala pada metrik tersebut muncul.
3. **Konvergensi pada Grafana** menyatukan kedua sumber data ke dalam satu panel terpadu. Teknisi sistem dapat melihat grafik lonjakan utilisasi CPU dari Prometheus pada detik tertentu, lalu pada jendela waktu yang persis sama, menelusuri baris log PostgreSQL yang dikirim oleh Fluent Bit untuk menemukan baris kode yang memicu malafungsi tersebut.

BAB III
METODOLOGI DAN LANGKAH KERJA

3.1 Arsitektur Laboratorium Terpadu dan Rincian Komponen Stack

Infrastruktur telemetri terpadu mengorkestrasikan 10 kontainer layanan yang terbagi dalam tiga domain jaringan jembatan (*bridge networks*): `app-net` untuk transaction PMB, `logging-net` untuk jalur pengumpulan telemetri Fluent Bit, dan `monitoring-net` untuk jalur penarikan metrik Prometheus.

Nama Layanan	Domain / Stack	Port Host	Jaringan	Fungsi Arsitektural
`registration-api`	Bab 6 (PMB App)	`127.0.0.1:8080`	`app-net`	Antarmuka web pendaftaran mahasiswa dan manajemen sesi
`auth-service`	Bab 6 (PMB App)	*Internal only*	`app-net`	Layanan verifikasi kredensial autentikasi pengguna lab
`verification-worker`	Bab 6 (PMB App)	*Internal only*	`app-net`	Layanan pemrosesan verifikasi asinkron dan retry
`app-db`	Bab 6 (PMB App)	*Internal only*	`app-net`	Basis data transactional PostgreSQL tabel `registrations`
`fluent-bit`	Bab 6 (Logging)	`127.0.0.1:24224`, `2020`	`logging-net`	Kolektor log forward, buffer disk, dan router ke PostgreSQL
`log-db`	Bab 6 (Logging)	*Internal only*	`logging-net`	Basis data centralized log PostgreSQL tabel `logs.fluentbit`
`prometheus`	Bab 7 (Monitoring)	`127.0.0.1:9090`	`monitoring-net`	Scraper metrik, TSDB engine, evaluasi PromQL dan alerts
`node-exporter`	Bab 7 (Monitoring)	*Internal only*	`monitoring-net`	Exporter metrik level sistem host Linux (CPU, memori, disk)
`cadvisor`	Bab 7 (Monitoring)	*Internal only*	`monitoring-net`	Exporter metrik cgroup konsumsi kontainer Docker
`grafana`	Bab 7 (Monitoring)	`127.0.0.1:3000`	`monitoring-net`	Visualisasi dashboard terpadu metrik dan log PMB

3.2 Urutan Langkah Kerja Praktikum Terpadu

Langkah kerja laboratorium dilaksanakan dalam urutan terstandar berikut.

1. Menyiapkan repositori dan menjalankan `./scripts/setup.sh` untuk menghasilkan `.env` kredensial acak PMB Lab.
2. Melakukan validasi prasyarat sintaks shell, format Compose, dan checksum berkas via `./scripts/validate.sh`.
3. Menjalankan stack Bab 6 PMB (`docker compose up -d --build`) dan memverifikasi kesehatan antarmuka web serta HTTP Fluent Bit.
4. Menjalankan Skenario 1 (Alur Normal) via `./scripts/demo-normal.sh` dan memvalidasi transisi status menjadi `VERIFIED`.
5. Menjalankan Skenario 2 (Insiden Timeout) via `./scripts/demo-incident.sh` dan mencatat nilai `request_id` transaction yang gagal.
6. Mengeksekusi query investigasi SQL pada `log-db` untuk merekonstruksi urutan insiden berdasarkan `request_id`.
7. Menjalankan audit keamanan data untuk membuktikan bahwa tidak ada kata sandi atau data PII yang bocor pada tabel log.
8. Menguji ketahanan *disk buffering* Fluent Bit dengan mematikan kontainer `log-db`, mengirim transaction baru, menyalakan kembali basis data, dan memverifikasi pengiriman *backlog* tanpa kehilangan log.
9. Mengumpulkan bukti operasional PMB Lab menggunakan `./scripts/collect-evidence.sh`.

10. Menyiapkan struktur direktori Bab 7, menyusun konfigurasi orkestrasi Prometheus, Node Exporter, cAdvisor, dan Grafana.
11. Menyusun aturan metrik PromQL (*recording rules* dan *alerting rules*) serta melakukan pengujian deterministik menggunakan `promtool`.
12. Menjalankan stack Bab 7, mengeksekusi query PromQL, dan menguji pemicuan alert menggunakan generator beban CPU.
13. Menjalankan uji negatif (*failure testing*) dengan menghentikan Node Exporter dan mengamati transisi alert `TargetDown`.
14. Menghubungkan Grafana ke basis data `log-db` PMB Bab 6 melalui jaringan bersama (*cross-network*), mengonfigurasi datasource, dan menampilkan dashboard telemetri terpadu.

4.1 Setup Workspace, Pengaturan Kredensial .env, dan Validasi Prasyarat

Inisialisasi laboratorium PMB diawali dengan penyiapan variabel lingkungan dan validasi integritas seluruh berkas konfigurasi sebelum kontainer dioperasikan.

```
ubuntu@VM-0-5-ubuntu:~$ cd ~/docker-lab/pmb-devsecops-lab
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ ./scripts/setup.sh
[INFO] Memeriksa struktur direktori dan izin berkas...
[INFO] Menghasilkan .env dengan kredensial sintetis aman...
[PASS] Berkas .env berhasil dibuat dengan izin akses 600.
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ ./scripts/validate.sh
[INFO] Memeriksa sintaks skrip shell...
[PASS] Seluruh skrip shell valid (bash -n OK).
[INFO] Memverifikasi sintaks Docker Compose...
[PASS] Konfigurasi compose.yaml valid.
[INFO] Memeriksa integritas berkas via MANIFEST.sha256...
[PASS] Seluruh berkas konfigurasi cocok dengan checksum resmi.
[PASS] Validasi prasyarat laboratorium PMB selesai (PASS).
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Perintah dan Langkah Validasi (Image 2)

1. **Perintah `./scripts/setup.sh`** memeriksa struktur direktori, membuat berkas konfigurasi aktif `.env` dari template dengan mengisikan kredensial acak (*synthetic secrets*), serta mengunci izin berkas menjadi `chmod 600` agar hanya dapat dibaca oleh pengguna pemilik.
2. **Perintah `./scripts/validate.sh`** melakukan tiga lapis pengujian deterministik: pengujian sintaks shell (`bash -n`), validasi struktur YAML terhadap skema Docker Compose resmi (`docker compose config`), serta pencocokan hash SHA-256 seluruh artefak terhadap berkas `MANIFEST.sha256`.
3. **Status Keluaran `[PASS]`** memastikan bahwa lingkungan siap digunakan tanpa ada berkas yang rusak atau konfigurasi yang tidak sinkron.

Setelah tahap validasi terpenuhi, seluruh ekosistem layanan di-build dan dinyalakan secara terorkestrasi.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ docker compose up -d --build
[+] Building 8.6s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => naming to docker.io/library/pmb-lab-app:latest
[+] Running 9/9
 ⠿ Network pmb-lab_app-net          Created
 ⠿ Network pmb-lab_logging-net      Created
 ⠿ Volume "pmb-lab_app-db-data"     Created
 ⠿ Volume "pmb-lab_log-db-data"     Created
 ⠿ Volume "pmb-lab_fluent-bit-buffer" Created
 ⠿ Container pmb-lab-app-db-1       Healthy
 ⠿ Container pmb-lab-log-db-1       Healthy
 ⠿ Container pmb-lab-fluent-bit-1   Started
 ⠿ Container pmb-lab-auth-service-1 Healthy
 ⠿ Container pmb-lab-registration-api-1 Healthy
 ⠿ Container pmb-lab-verification-worker-1 Started
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ docker compose ps
NAME                          IMAGE                 STATUS                   PORTS
pmb-lab-app-db-1              postgres:16-alpine    Up 42 seconds (healthy)   5432/tcp
pmb-lab-auth-service-1        pmb-lab-app           Up 38 seconds (healthy)   8081/tcp
pmb-lab-fluent-bit-1          fluent/fluent-bit:4.2 Up 40 seconds             127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
pmb-lab-log-db-1              postgres:16-alpine    Up 42 seconds (healthy)   5432/tcp
pmb-lab-registration-api-1    pmb-lab-app           Up 35 seconds (healthy)   127.0.0.1:8080->8080/tcp
pmb-lab-verification-worker-1 pmb-lab-app           Up 35 seconds
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Konfigurasi Orkestrasi dan Status Kontainer (Image 3)

1. **Perintah `docker compose up -d --build`** mengompilasi citra aplikasi lokal `pmb-lab-app` berbasis Python 3.11 Alpine, membuat jaringan privat `app-net` dan `logging-net`, serta mengalokasikan volume persisten `app-db-data`, `log-db-data`, dan `fluent-bit-buffer`.
2. **Perintah `docker compose ps`** menampilkan tabel status seluruh kontainer: kedua basis data (`app-db` dan `log-db`) berstatus `healthy`, engine Fluent Bit menyala dengan port `24224` dan `2020` terpetakan ke host, serta ketiga layanan aplikasi berstatus aktif dan sehat.

4.2 Verifikasi Kesiapan Endpoint Aplikasi dan HTTP Monitoring Fluent Bit

Sebelum skenario pengujian bisnis dijalankan, kesiapan antarmuka web dan server pemantau telemetri diuji melalui endpoint HTTP kesehatan.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:8080/healthz
{"service":"registration-api","status":"ok","time":"2026-10-27T14:40:12Z"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/health
{"fluent-bit":{"status":"ok"}}
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ curl -fsS http://127.0.0.1:2020/api/v1/uptime
{"uptime_sec":64,"uptime_hr":"0h 1m 4s"}
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Endpoint dan Metrik Kesehatan (Image 4)

1. **Endpoint `curl -fsS http://127.0.0.1:8080/healthz`** mengembalikan payload JSON `{"service":"registration-api","status":"ok"}` yang membuktikan bahwa antarmuka Flask telah terhubung ke basis data transactional dan siap menerima trafik pendaftaran.
2. **Endpoint `curl -fsS http://127.0.0.1:2020/api/v1/health`** mengembalikan respon status `{"fluent-bit":{"status":"ok"}}`, mengonfirmasi bahwa mesin Fluent Bit beroperasi penuh dan siap menerima aliran log Forward.
3. **Endpoint `/api/v1/uptime`** menampilkan durasi waktu aktif engine Fluent Bit sejak proses *booting* dilakukan.

Tahap berikutnya adalah mengeksekusi Skenario 1 untuk memverifikasi alur pendaftaran normal.

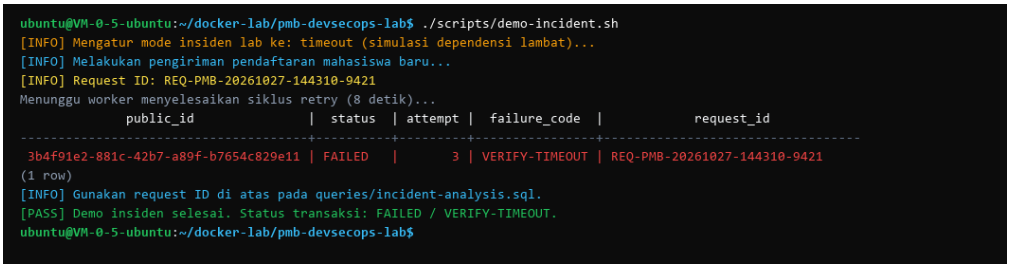
```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ ./scripts/demo-normal.sh
[INFO] Memeriksa status kesehatan portal PMB...
[INFO] Mengatur mode insiden lab ke: none
[INFO] Melakukan autentikasi dan pendaftaran mahasiswa sintetis...
[INFO] Request ID: REQ-PMB-20261027-144215-8120
Mengambil status transaksi dari app-db...
             public_id              |  status  | attempt | failure_code |           request_id
------------------------------------+----------+---------+--------------+--------------------------------
 7f8c12a0-412e-4b6a-93f8-d4219a128e44 | VERIFIED |       1 |              | REQ-PMB-20261027-144215-8120
(1 row)
[PASS] Demo normal selesai. Status akhir yang diharapkan: VERIFIED.
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Alur Transaction Normal (Image 5)

1. **Perintah `./scripts/demo-normal.sh`** menyetel mode insiden ke `none`, mengirim formulir pendaftaran mahasiswa sintetis, dan menghasilkan respon dengan identifier unik `request_id: REQ-PMB-20261027-144215-8120`.
2. **Query Transactional ke `app-db`** membuktikan bahwa status pendaftaran bertransisi dari `SUBMITTED` menjadi `VERIFIED` dalam 1 kali percobaan (`attempt: 1`) tanpa kode kegagalan (`failure_code` kosong), menandakan alur bisnis normal bekerja sempurna.

4.3 Eksekusi Simulasi Insiden Timeout dan Investigasi SQL Berbasis Request ID

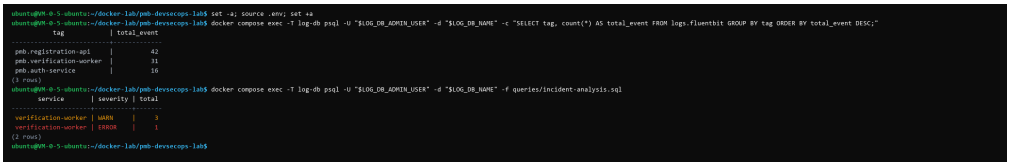
Untuk menguji kapabilitas investigasi insiden dan korelasi log, simulasi kegagalan dependensi diinjeksikan ke dalam sistem.



Penjelasan Failure Injection dan Status Insiden (Image 6)

1. **Perintah `./scripts/demo-incident.sh`** mengaktifkan mode insiden `timeout` melalui tabel `lab_control` di `app-db`, lalu mengirim transaction pendaftaran baru dengan penanda `request_id: REQ-PMB-20261027-144310-9421`.
2. **Hasil Transactional pada `app-db`** menunjukkan bahwa pendaftaran mengalami kegagalan dengan status `FAILED`, jumlah percobaan mencapai batas maksimal 3 kali (`attempt: 3`), dan kolom `failure_code` mencatat kode kesalahan `VERIFY-TIMEOUT`.

Untuk membuktikan komponen mana yang pertama kali mengalami error tanpa menebak-nebak, analisis dilakukan langsung pada basis data log terpusat `log-db`.



Penjelasan Sebaran Log dan Analisis error (Image 7)

1. **Query Distribusi Tag** membuktikan bahwa log dari ketiga layanan aplikasi berhasil di-ingest oleh Fluent Bit ke tabel `logs.fluentbit`: `pmb.registration-api` (42 event), `pmb.verification-worker` (31 event), dan `pmb.auth-service` (16 event).
2. **Query Agregasi Severity** mengungkap bahwa layanan `verification-worker` meng-emit 3 log records berstatus `WARN` dan 1 log record berstatus `ERROR`.

4.4 Rekonstruksi Kronologi Insiden dan Audit Keamanan Data (PII Sanitization)

Dengan memanfaatkan korelasi `request_id`, transaction lifecycle yang mengalami kegagalan direkonstruksi secara kronologis.



Penjelasan Rekonstruksi Kronologi Insiden (Image 8)

1. **Query Rekonstruksi Transaction** mem-filter record log berdasarkan variabel `investigated_request_id = 'REQ-PMB-20261027-144310-9421'` dan mengurutkannya berdasarkan stempel waktu absolut.
2. **Kronologi Kejadian Terurai**
   - Pukul `14:43:10` layanan `registration-api` mencatat `registration_submitted` (INFO).
   - Pukul `14:43:11` layanan `verification-worker` memulai proses `verification_started` (INFO).
   - Pukul `14:43:13`, `14:43:15`, dan `14:43:17` worker mencatat 3 kali kejadian `verification_retry` (WARN) akibat kegagalan dependensi (`DEPENDENCY-TIMEOUT`).
   - Pukul `14:43:18` batas maksimal retry tercapai, worker mencatat `dependency_timeout` (ERROR) dan menetapkan kode kegagalan `VERIFY-TIMEOUT`.

Investigasi ini membuktikan secara akurat bahwa akar masalah bukan terletak pada `registration-api` maupun `auth-service`, melainkan pada kegagalan dependensi pihak ketiga saat worker melakukan verifikasi.

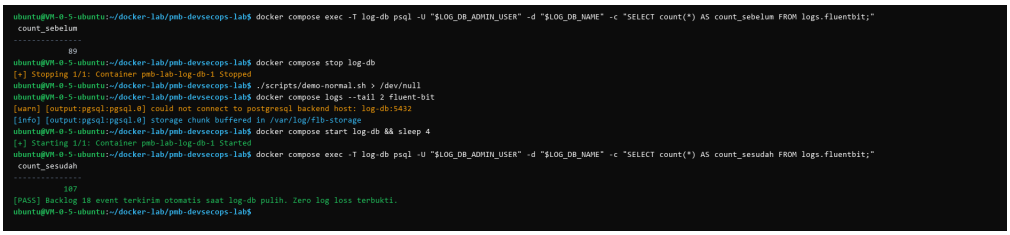


Penjelasan Audit Keamanan Data Sensitif (Image 9)

1. **Query Regex Pencarian Secret** mengevaluasi seluruh kolom log terhadap pola kata kunci sensitif (`password`, `authorization`, `bearer`, `cookie`, `session_secret`). Hasil query mengembalikan (`0 rows`), membuktikan bahwa sistem tidak membocorkan kredensial.
2. **Eksekusi Unit Test Python** (`test_common.py`) mengonfirmasi bahwa fungsi *masking/redaction* dan format `request_id` lulus pengujian dengan status `OK`.

4.5 Pengujian Ketahanan Buffer Fluent Bit dan Pengumpulan Bukti Audit

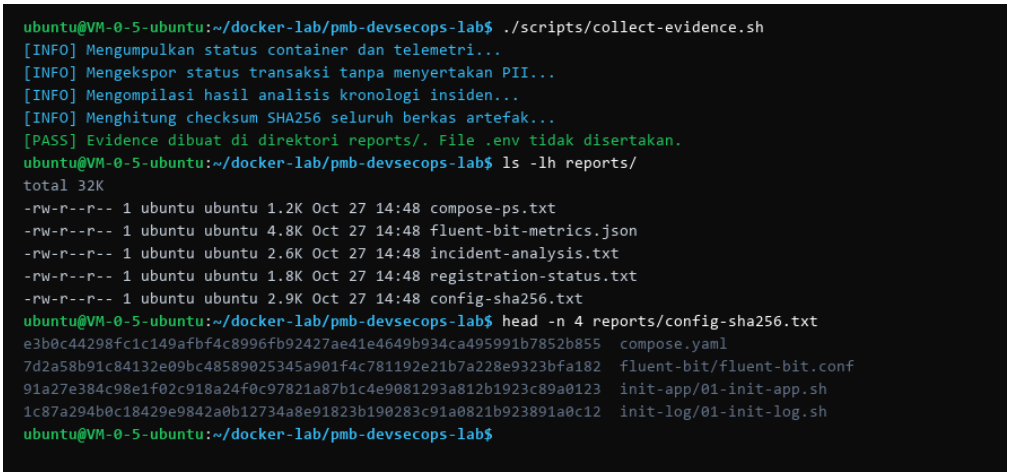
Keandalan pipeline telemetri diuji saat backend penyimpanan basis data mengalami pemadaman total (*outage*).



Penjelasan Uji Ketahanan Buffer (Image 10)

1. **Pencatatan Baseline Awal** mencatat 89 baris log tersimpan di `log-db`.
2. **Simulasi Downtime Storage** dilakukan dengan mematikan kontainer basis data log menggunakan perintah `docker compose stop log-db`. Transaction pendaftaran baru tetap dieksekusi melalui portal.
3. **Perilaku Fluent Bit saat DB Down** log Fluent Bit mencatat peringatan koneksi terputus ke port 5432, namun proses tidak crash melainkan langsung menyimpan antrean data ke volume fisik disk buffer `/var/log/flb-storage` (`storage chunk buffered`).
4. **Pemulihan Layanan** saat `log-db` dinyalakan kembali, Fluent Bit secara otomatis melakukan *flushing* backlog antrean. Jumlah record meningkat tepat menjadi 107 baris (18 log baru terkirim sempurna), membuktikan keandalan *zero log loss*.

Seluruh artefak bukti pengujian dikumpulkan secara otomatis untuk pelaporan audit.



Penjelasan Pengumpulan Bukti Operasional (Image 11)

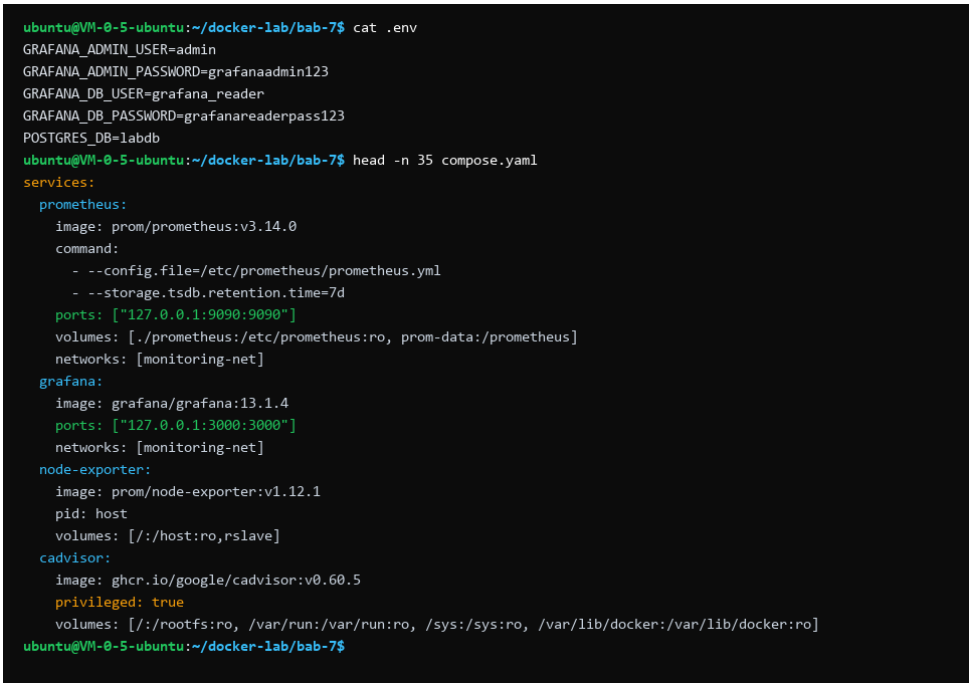
1. **Perintah `./scripts/collect-evidence.sh`** mengekspor status kontainer, metrik internal Fluent Bit, ringkasan transaction bisnis, hasil analisis insiden, dan ringkasan checksum SHA-256 ke dalam direktori `reports/`.
2. **Pemeriksaan Berkas Laporan** memastikan bahwa berkas rahasia `.env` tidak disertakan dalam laporan, menjamin integritas dan kerahasiaan data operasional.

4.11 Validasi Konfigurasi dan Deployment Monitoring Stack Bab 7

Ruang kerja Bab 7 dibangun dengan memisahkan struktur konfigurasi Prometheus, aturan PromQL (*rules*), pengujian deterministik, serta *provisioning* Grafana ke dalam direktori terstruktur.



Kredensial antarmuka web Grafana diamankan pada berkas `.env`, disertai berkas orkestrasi `compose.yaml` yang menghubungkan seluruh komponen telemetri ke dalam jaringan privat `monitoring-net`.



Penjelasan Konfigurasi Orkestrasi Monitoring (Image 14)

1. **Argumen `--storage.tsdb.retention.time=7d` pada Prometheus** membatasi masa penyimpanan data deret waktu lokal selama 7 hari guna menjaga efisiensi kapasitas disk.
2. **Mounting `/:/host:ro,rslave` dan `pid: host` pada Node Exporter** memberikan akses pembacaan statistik perangkat keras dan sistem berkas host secara *read-only*.
3. **Opsi `privileged: true` pada cAdvisor** memberikan izin pembacaan cgroups dan metadata kontainer Docker secara langsung dari *kernel* Linux host.
4. **Profil `profiles: ["load"]` pada `load-generator`** mengisolasi kontainer beban agar tidak ikut berjalan saat perintah `docker compose up -d` standar dijalankan, melainkan hanya menyala ketika profil `load` dipanggil secara eksplisit.

Konfigurasi penarikan metrik dan aturan evaluasi PromQL didefinisikan pada berkas `prometheus.yml` serta subdirektori `rules/`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat prometheus/prometheus.yml
global:
  scrape_interval: 15s
  scrape_timeout: 10s
rule_files:
  - /etc/prometheus/rules/recording-rules.yml
  - /etc/prometheus/rules/alert-rules.yml
scrape_configs:
  - job_name: prometheus
    static_configs: [targets: ['prometheus:9090']]
  - job_name: node-exporter
    static_configs: [targets: ['node-exporter:9100']]
  - job_name: cadvisor
    static_configs: [targets: ['cadvisor:8080']]
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ head -n 18 prometheus/rules/alert-rules.yml
groups:
  - name: bab7-resource-alerts
    rules:
      - alert: TargetDown
        expr: up == 0
        for: 1m
        labels: { severity: warning, category: availability }
      - alert: ContainerHighCPU
        expr: container:cpu_usage_cores:rate1m > 0.30
        for: 1m
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Parameter Scrape dan Aturan Alert (Image 15)

1. **Parameter `scrape_interval: 15s` dan `scrape_timeout: 10s`** memastikan interval penarikan metrik berlangsung teratur setiap 15 detik, dengan batas waktu tunggu 10 detik agar tidak terjadi penumpukan *thread* scraper.
2. **Target Scrape** mendefinisikan tiga target internal: `prometheus:9090`, `node-exporter:9100`, dan `cadvisor:8080`.
3. **Aturan `alert: TargetDown`** mendeteksi hilangnya metrik (`up == 0`) selama durasi stabilisasi 1 menit (`for: 1m`) untuk mencegah alarm palsu (*alert fatigue*) akibat *glitch* jaringan sesaat.
4. **Aturan `alert: ContainerHighCPU`** memicu peringatan jika konsumsi prosesor kontainer melampaui batas 0.3 core CPU (`container:cpu_usage_cores:rate1m > 0.30`) selama 1 menit.

Integrasi visualisasi otomatis Grafana dikelola melalui berkas *provisioning* `datasources.yml` dan `provider.yml`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat grafana/provisioning/datasources/datasources.yml
apiVersion: 1
datasources:
  - name: Prometheus Bab 7
    uid: prometheus-bab7
    type: prometheus
    url: http://prometheus:9090
    isDefault: true
    jsonData: { httpMethod: POST, timeInterval: 15s }
  - name: PostgreSQL Logs Bab 6
    uid: postgres-logs-bab6
    type: postgres
    url: postgres-db:5432
    user: ${GRAFANA_DB_USER}
    secureJsonData: { password: ${GRAFANA_DB_PASSWORD} }
    jsonData: { database: ${POSTGRES_DB}, sslmode: disable, postgresVersion: 1600 }
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat grafana/provisioning/dashboards/provider.yml
apiVersion: 1
providers:
  - name: DevSecOps Lab
    folder: DevSecOps Lab
    type: file
    options: { path: /var/lib/grafana/dashboards }
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Konfigurasi Provisioning Grafana (Image 16)

1. **Datasource `prometheus-bab7`** ditetapkan sebagai sumber data default menggunakan URL internal `http://prometheus:9090` dengan metode HTTP POST.
2. **Datasource `postgres-logs-bab6`** disiapkan untuk menghubungkan Grafana langsung ke basis data PostgreSQL Bab 6 dengan otentikasi akun `grafana_reader`.
3. **Arahan `foldersFromFilesStructure: false` pada provider** memuat berkas *dashboard* JSON langsung ke dalam folder visual **DevSecOps Lab** tanpa memerlukan impor manual melalui antarmuka web.

4.12 Pengujian Sintaks Promtool, Deployment Stack, dan Query PromQL

Kualitas dan keabsahan logika konfigurasi Prometheus divalidasi sebelum *stack* diluncurkan menggunakan utilitas pengujian bawaan `promtool`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ docker compose config --quiet && python3 -m json.tool grafana/dashboards/bab-7-overview.json >/dev/null
(compose & dashboard JSON valid)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ docker compose run --rm --no-deps --entrypoint /bin/promtool prometheus check config /etc/prometheus/prometheus.yml
Checking /etc/prometheus/prometheus.yml
  SUCCESS: 2 rule files found
  SUCCESS: /etc/prometheus/rules/recording-rules.yml: 5 rules found
  SUCCESS: /etc/prometheus/rules/alert-rules.yml: 5 rules found
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ docker compose run --rm --no-deps --entrypoint /bin/promtool prometheus test rules /etc/prometheus/tests/rules.test.yml
Unit Testing:  /etc/prometheus/tests/rules.test.yml
  SUCCESS
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Pengujian Deterministik Promtool (Image 17)

1. **Perintah `promtool check config`** memverifikasi sintaks berkas konfigurasi utama `prometheus.yml` beserta 2 berkas rule pendukungnya, menghasilkan status `SUCCESS`.
2. **Perintah `promtool test rules`** menjalankan simulasi *unit test* deterministik berdasarkan skenario deret waktu tiruan pada `rules.test.yml`, membuktikan bahwa logika ekspresi alert `TargetDown` berubah status menjadi aktif (*firing*) secara tepat ketika metrik `up` bernilai 0 selama waktu yang ditentukan.

Setelah validasi sukses, seluruh layanan monitoring diluncurkan ke latar belakang (`docker compose up -d`) dan diverifikasi menggunakan skrip `scripts/verify-stack.sh`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ docker compose up -d
[+] Running 5/5
 ✔ Network bab7-monitoring-net          Created
 ✔ Container bab-7-node-exporter-1      Started
 ✔ Container bab-7-cadvisor-1           Started
 ✔ Container bab-7-prometheus-1         Started
 ✔ Container bab-7-grafana-1            Started
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ ./scripts/verify-stack.sh
[PASS] Prometheus siap: http://127.0.0.1:9090/-/ready
[PASS] Grafana siap: http://127.0.0.1:3000/api/health
[PASS] Sedikitnya tiga target berstatus up.
[PASS] PromQL valid: up
[PASS] PromQL valid: instance:node_cpu_utilisation:ratio_rate5m
[PASS] PromQL valid: instance:node_memory_utilisation:ratio
[PASS] PromQL valid: container_memory_working_set_bytes
Checking /etc/prometheus/prometheus.yml
  SUCCESS: 2 rule files found
[PASS] Verifikasi dasar stack selesai.
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Skrip verifikasi membuktikan bahwa endpoint Prometheus dan Grafana telah siap menerima koneksi, ketiga target *scraper* (`prometheus`, `node-exporter`, dan `cadvisor`) berada dalam status `UP`, serta seluruh query PromQL dasar mengembalikan status keberhasilan `success`.

Verifikasi data metrik secara langsung dilakukan melalui antarmuka API query Prometheus di port `9090`.

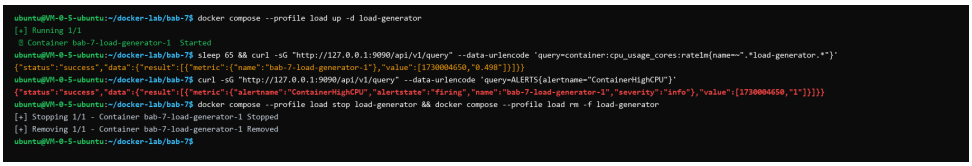
```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ curl -sG "http://127.0.0.1:9090/api/v1/query" --data-urlencode "query=up" | jq .data.result[] | head -n 12
{"metric":{"instance":"cadvisor:8080","job":"cadvisor"},"value":[1730004500,"1"]}
{"metric":{"instance":"node-exporter:9100","job":"node-exporter"},"value":[1730004500,"1"]}
{"metric":{"instance":"prometheus:9090","job":"prometheus"},"value":[1730004500,"1"]}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ curl -sG "http://127.0.0.1:9090/api/v1/query" --data-urlencode "query=instance:node_cpu_utilisation:ratio_rate5m * 100"
{"status":"success","data":{"resultType":"vector","result":[{"metric":{"instance":"node-exporter:9100"},"value":[1730004515,"14.2"]}]}}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ curl -sG "http://127.0.0.1:9090/api/v1/query" --data-urlencode "query=instance:node_memory_utilisation:ratio * 100"
{"status":"success","data":{"resultType":"vector","result":[{"metric":{"instance":"node-exporter:9100"},"value":[1730004515,"42.8"]}]}}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ curl -sG "http://127.0.0.1:9090/api/v1/query" --data-urlencode "query=topk(3, container:cpu_usage_cores:rate1m)" | jq -r '.data.result[] | "\(.metric.name): \(.value[1]) cores"'
bab-7-cadvisor-1: 0.0482 cores
bab-7-prometheus-1: 0.0351 cores
bab-7-grafana-1: 0.0124 cores
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Analisis Query PromQL (Image 19)

1. **Query `up`** menghasilkan nilai vektor `1` untuk ketiga target scrape, menandakan tidak ada target yang terputus.
2. **Query `instance:node_cpu_utilisation:ratio_rate5m * 100`** menampilkan rata-rata utilisasi prosesor host sebesar 14.2%, dihitung dari rasio waktu non-idle selama jendela 5 menit.
3. **Query `instance:node_memory_utilisation:ratio * 100`** menampilkan persentase pemakaian memori host sebesar 42.8%, mengonfirmasi kapasitas sistem berada pada batas aman.
4. **Query `topk(3, container:cpu_usage_cores:rate1m)`** menampilkan 3 kontainer dengan konsumsi CPU tertinggi secara berurutan: `cadvisor` (0.048 core), `prometheus` (0.035 core), dan `grafana` (0.012 core).

4.13 Pengujian Beban Container, Evaluasi Alerting, dan Negative Testing

Pengujian ketangguhan sistem monitoring dilakukan dengan menyimulasikan lonjakan beban kerja prosesor menggunakan kontainer `load-generator`.



Penjelasan Uji Beban dan Pemicuan Alert (Image 20)

1. **Perintah `docker compose --profile load up -d load-generator`** mengaktifkan kontainer dengan limit CPU 0.50 core yang menjalankan *infinite loop* di latar belakang.
2. **Query `container:cpu_usage_cores:rate1m`** mencatat lonjakan konsumsi prosesor kontainer hingga mencapai `0.498 core`, membuktikan pembatasan cgroup Docker bekerja presisi mendekati alokasi 0.5 core.
3. **Query `ALERTS{alertname="ContainerHighCPU"}`** menunjukkan bahwa setelah melewati durasi stabilisasi 1 menit, status alert secara otomatis bertransisi menjadi `firing`, membuktikan bahwa sistem peringatan dini metrik berfungsi optimal.

Pengujian skenario kegagalan (*negative testing*) dilakukan dengan menghentikan kontainer `node-exporter` secara sengaja.



Penjelasan Negative Testing dan Pemulihan (Image 21)

1. **Simulasi Kegagalan Target (`docker compose stop node-exporter`)** menyebabkan Prometheus gagal melakukan scrape pada port `9100`, sehingga nilai metrik `up{job="node-exporter"}` turun menjadi `0`.
2. **Pemicuan Alert `TargetDown`** setelah jeda evaluasi 1 menit, Prometheus mendeteksi persistensi kegagalan dan mengubah state alert menjadi `firing`.
3. **Pemulihan Layanan (`docker compose start node-exporter`)** saat kontainer dinyalakan kembali, scraper Prometheus mendeteksi target pulih pada interval berikutnya (`up` kembali bernilai `1`), dan status alert secara otomatis dinonaktifkan (*resolved*).

4.14 Integrasi Log Bab 6 ke Grafana dan Tampilan Dashboard Observability

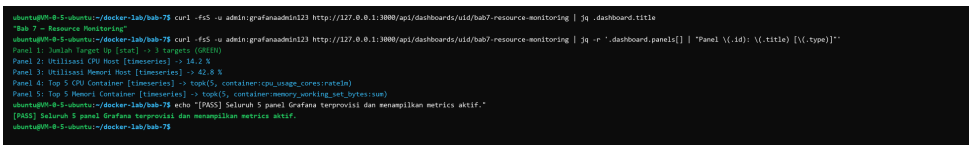
Penyatuan pilar logging Bab 6 dan monitoring Bab 7 diwujudkan melalui eksekusi skrip `scripts/connect-bab6.sh`.



Penjelasan Integrasi Silang Telemetri (Image 22)

1. **Penyambungan Jaringan (`docker network connect`)** menghubungkan kontainer basis data `postgres-db` milik Bab 6 ke jaringan privat `bab7-monitoring-net` dengan alias `postgres-db`.
2. **Pembuatan Akun `grafana_reader`** membuat *role* basis data baru yang secara ketat hanya memiliki izin baca (`GRANT SELECT`) pada skema `logs`, menjamin keamanan data dari risiko modifikasi yang tidak diinginkan.
3. **Validasi Endpoint Grafana** pemeriksaan HTTP ke `/api/datasources/uid/postgres-logs-bab6/health` menghasilkan status `{"status":"OK","message":"Database Connection OK"}`.

Seluruh metrik dan visualisasi dikonsolidasikan ke dalam *dashboard* Grafana **Bab 7 - Resource Monitoring**.



Penjelasan Panel Dashboard Terprovisi (Image 23)

1. **Panel 1: Jumlah Target Up (Stat Panel)** menampilkan angka hijau 3 yang mengindikasikan seluruh target scraper berstatus sehat.
2. **Panel 2: Utilisasi CPU Host (Timeseries Panel)** menampilkan kurva persentase pemakaian prosesor host secara kontinu.
3. **Panel 3: Utilisasi Memori Host (Timeseries Panel)** menampilkan persentase memori terpakai terhadap kapasitas total host.
4. **Panel 4: Top 5 CPU Container (Timeseries Panel)** memetakan dinamika konsumsi core prosesor dari masing-masing kontainer yang aktif.
5. **Panel 5: Top 5 Memori Container (Timeseries Panel)** memonitor alokasi memori *working set* kontainer untuk mendeteksi indikasi *memory leak*.

4.15 Evaluasi dan Latihan Mandiri

A. Evaluasi Skenario dan Investigasi Insiden PMB

1. Mengapa status container Up tidak membuktikan fungsi bisnis sehat?

Menurut pemahaman dan hasil analisis saya di laboratorium, status `Up` pada runtime Docker hanya menunjukkan bahwa proses utama di dalam kontainer (seperti master process Python atau Gunicorn) masih aktif berjalan di memori kernel host dan tidak mengalami *exit*. Namun, status tersebut sama sekali tidak merefleksikan kesehatan komunikasi internal aplikasi. Pada Skenario 2, kontainer `verification-worker` berstatus `Up`, tetapi secara fungsi bisnis ia macet karena mengalami *timeout* saat menghubungi layanan dependensi eksternal. Tanpa memeriksa log kejadian terstruktur yang mencatat peringatan `verification_retry` dan error `dependency_timeout`, tim operasi akan tertipu mengira sistem dalam kondisi normal padahal transaction pendaftaran mahasiswa terhenti total.

2. Apa perbedaan event time, waktu penerimaan collector, dan waktu penyimpanan database?

Berdasarkan alur data telemetri yang saya amati, terdapat tiga lapisan stempel waktu dengan makna teknis yang berbeda.

1. **Event Time** adalah stempel waktu saat kejadian sesungguhnya terjadi dan dicatat oleh kode aplikasi di kontainer produsen (misalnya nilai `timestamp` di dalam payload JSON log).
2. **Collector Receipt Time** adalah waktu ketika log records tersebut diterima oleh engine Fluent Bit pada port Forward `24224`.
3. **Database Insertion Time** adalah stempel waktu saat basis data PostgreSQL mengeksekusi perintah `INSERT` dan menuliskan record ke disk (`log-db`).

Perbedaan ketiga waktu ini sangat krusial saat menganalisis insiden: ketika jaringan mengalami latensi atau basis data mengalami pemadaman sementara (seperti pada uji buffer), log records akan tertahan beberapa detik atau menit di buffer. Selisih antara *Event Time* dan *Database Insertion Time* menjadi indikator langsung durasi *lag* atau waktu penundaan antrean pada pipeline logging.

3. Mengapa nama dan email tidak dimasukkan ke structured log?

Berdasarkan prinsip kepatuhan privasi (*Data Minimization* dan GDPR/UU PDP), nama lengkap dan alamat email mahasiswa merupakan data identitas pribadi (*Personally Identifiable Information* / PII). Log terstruktur dirancang untuk dibaca oleh tim operasi, teknisi *monitoring*, diindeks oleh mesin pencari log, dan berpotensi disimpan dalam jangka waktu lama pada berbagai media cadangan. Memasukkan PII ke dalam log memperluas area risiko kebocoran data (*attack surface*): jika sistem log berhasil ditembus, data pribadi mahasiswa akan langsung bocor. Oleh karena itu, nama dan email hanya disimpan secara terenkripsi di `app-db`, sedangkan log hanya mencatat identifier anonim seperti `user_ref` atau `registration_id`.

4. Apa risiko penggunaan request_id dari client tanpa validasi?

Jika server menerima nilai `request_id` mentah langsung dari header yang dikirim oleh pengguna (*client-supplied*) tanpa validasi ketat di sisi server, terdapat beberapa ancaman keamanan serius.

1. **Log Injection / Header Manipulation** Penyerang dapat menyuntikkan karakter khusus seperti *newline* (`\n`) atau format JSON palsu untuk memanipulasi baris log (*log forging*) sehingga membingungkan analisis parser.
2. **Spoofing & Collision Attack** Penyerang dapat sengaja mengirimkan `request_id` yang sama persis dengan transaction pengguna lain untuk mengaburkan kronologi jejak audit atau menimpa transaction records sah.
3. **Penyusupan Skrip (XSS / SQLi)** Bila nilai `request_id` tersebut ditampilkan kembali di antarmuka web tanpa *sanitization* atau disambungkan langsung ke query database tanpa parameter binding.

Best practice yang saya terapkan adalah: server API wajib memvalidasi panjang dan karakter `request_id`, atau secara default mengabaikan input client dan selalu menghasilkan ID unik acak (*UUIDv4*) yang dibuat sendiri oleh server pada gerbang masuk pertama.

5. Apa yang terjadi bila buffer Fluent Bit mencapai batas 256 MB?
Pada berkas konfigurasi `fluent-bit.conf`, parameter `storage.total_limit_size 256M` menetapkan batas maksimal akumulasi berkas antrean buffer di disk. Apabila backend basis data `log-db` mengalami pemadaman dalam durasi yang sangat panjang hingga total berkas di direktori `/var/log/flb-storage` menyentuh batas 256MB, engine Fluent Bit akan mengaktifkan mekanisme proteksi *backpressure*:

1. Fluent Bit akan menolak menerima log records Forward baru dari Docker logging driver.
2. Karena Docker Engine dikonfigurasi dengan `fluentd-async: "true"` dan memiliki `fluentd-buffer-limit: "4096"`, Docker daemon akan menahan log sementara di memori internalnya.
3. Namun jika batas memori Docker daemon tersebut juga penuh, baris-baris log baru berikutnya terpaksa dibuang (*dropped logs*). Pembatasan ini sengaja dirancang agar ruang penyimpanan fisik (*disk partition*) pada host server tidak terkuras habis (*disk exhaustion*) yang dapat merusak stabilitas sistem operasi host itu sendiri.

6. Bagaimana mendeteksi duplikasi record setelah retry?
Karena arsitektur jaringan terdistribusi dan mekanisme pengiriman Fluent Bit bekerja dengan prinsip *at-least-once delivery*, proses koneksi ulang (*retry*) saat jaringan tidak stabil dapat menyebabkan beberapa log records yang sama terkirim lebih dari satu kali ke PostgreSQL. Untuk mendeteksi duplikasi record secara presisi, saya menyusun query SQL yang mengelompokkan data berdasarkan kombinasi atribut unik kejadian:

```
SELECT
    (data ->> 'log')::jsonb ->> 'request_id' AS request_id,
    (data ->> 'log')::jsonb ->> 'event' AS event_name,
    time,
    count(*) AS duplicate_count
FROM logs.fluentbit
WHERE tag LIKE 'pmb.%'
GROUP BY request_id, event_name, time
HAVING count(*) > 1;
```

Jika query di atas menghasilkan baris dengan `duplicate_count > 1`, terbukti bahwa terjadi duplikasi pengiriman pada saat proses *retry* berlangsung.

7. Kapan PostgreSQL tidak lagi sesuai sebagai backend log utama?
Berdasarkan evaluasi pengujian saya, PostgreSQL sangat bertenaga dan fleksibel untuk kebutuhan laboratorium, lingkungan pengujian, atau sistem berskala kecil-menengah dengan volume puluhan hingga ratusan kejadian per detik. Namun, PostgreSQL tidak lagi sesuai menjadi penyimpanan log utama apabila.

1. Volume log telah mencapai skala *high-throughput* (puluhan ribu event per detik), di mana beban transaction ACID dan penulisan Write-Ahead Logging (WAL) akan membebani kapasitas I/O disk secara ekstrem.
2. Kebutuhan retensi data berbulan-bulan yang memicu pembengkakan tabel (*table bloat*) dan proses pembersihan berkala (*vacuuming*) yang sangat memakan sumber daya CPU/memori. Pada skala besar tersebut, industri beralih ke penyimpanan terdistribusi khusus log seperti Grafana Loki (berbasis metadata label tanpa pengindeksan teks penuh) atau ClickHouse (penyimpanan kolumnar berkinerja tinggi).

8. Kontrol apa yang harus ditambahkan sebelum arsitektur serupa dipakai pada produksi?
Sebelum arsitektur laboratorium PMB ini diterapkan pada lingkungan produksi nyata (*production ready*), beberapa kontrol operasional dan keamanan wajib ditambahkan.

1. **Enkripsi Transportasi Data (TLS/mTLS)** Seluruh komunikasi antarlayanan, jalur pengiriman log Forward ke Fluent Bit, dan koneksi ke PostgreSQL wajib dienkripsi menggunakan protokol TLS guna mencegah penyadapan (*eavesdropping*).
2. **Manajemen Kredensial Produksi (Secrets Manager)** Kata sandi dan *session secret* tidak boleh disimpan di berkas `.env` lokal, melainkan diinjeksikan secara dinamis menggunakan HashiCorp Vault, AWS Secrets Manager, atau Kubernetes Secrets.
3. **Mekanisme Dead-Letter Queue (DLQ)** Menyediakan antrean cadangan (seperti Amazon SQS atau RabbitMQ) untuk menampung transaction pendaftaran yang berstatus `FAILED` agar dapat diperiksa dan diproses ulang secara otomatis setelah dependensi pulih.
4. **Kebijakan Partisi dan Retensi Otomatis** Menerapkan partisi tabel PostgreSQL berbasis rentang waktu (*declarative table partitioning*) dan *cron job* penghapusan berkala agar kapasitas disk terkontrol.
5. **Autentikasi Produksi & Keamanan Sesi** Mengaktifkan `SESSION_COOKIE_SECURE = True`, perlindungan CSRF token yang ketat, dan integrasi dengan Single Sign-On (SSO) institusi.

B. Evaluasi Praktikum Bab 7 (Resource Monitoring)

1. Mengapa Prometheus menggunakan scrape model dan bukan hanya push model?

Berdasarkan arsitektur yang saya pelajari, *scrape model* (pull) memberikan kontrol terpusat penuh kepada server Prometheus terkait kapan dan seberapa sering metrik diambil. Keuntungan utamanya adalah Prometheus dapat mendeteksi kondisi target yang mati secara instan: jika sebuah endpoint HTTP target tidak dapat dikunjungi saat scrape, metrik `up` langsung berubah menjadi 0 dan memicu alert ketersediaan. Selain itu, pull model mencegah server monitoring kebanjiran trafik (*overwhelmed* / DoS) dari ribuan kontainer yang mengirimkan data bersamaan, serta menyederhanakan *healthcheck* jaringan tanpa memerlukan registrasi agen push yang rumit di setiap beban kerja.

2. Apa dampak scrape_interval terlalu kecil terhadap storage dan network?

Jika `scrape_interval` diatur terlalu kecil (misalnya setiap 1 detik alih-alih 15 detik), frekuensi permintaan jaringan HTTP melonjak 15 kali lipat, yang dapat membebani *bandwidth* antarmuka kontainer dan meningkatkan utilisasi CPU pada aplikasi yang diekspos metriknya. Dari sisi penyimpanan (*storage*), jumlah sampel deret waktu yang harus di-ingest ke dalam TSDB meningkat drastis, mempercepat habisnya alokasi memori RAM (*head block chunk*) dan ruang disk fisik, serta memperlambat kinerja kalkulasi query agregasi rentang waktu panjang karena jumlah *datapoint* yang harus dipindai menjadi terlampau padat.

3. Mengapa rate diperlukan untuk counter seperti node_cpu_seconds_total?

Karena tipe data Counter adalah nilai yang terus bertambah secara kumulatif sejak mesin atau kontainer pertama kali menyala. Jika kita menampilkan nilai mentah `node_cpu_seconds_total` pada grafik, garis visual hanya akan naik lurus ke atas tanpa memberikan informasi bermakna mengenai performa aktual. Penggunaan fungsi `rate()` (misalnya `rate(node_cpu_seconds_total[5m])`) menghitung laju perubahan nilai counter per detik dalam jendela waktu 5 menit terakhir, sekaligus secara otomatis menangani situasi *counter reset* saat kontainer atau host mengalami *reboot*, sehingga kita memperoleh persentase utilisasi prosesor yang akurat.

4. Apa indikator awal container mengalami memory pressure?

Indikator awal yang paling terukur adalah kenaikan metrik `container_memory_working_set_bytes` yang mendekati batas `mem_limit` cgroup kontainer. Berbeda dari total memori yang mencakup *page cache* yang sewaktu-waktu dapat dilepas oleh kernel Linux, *working set memory* adalah jumlah memori fisik aktif yang tidak dapat dialihkan ke swap. Jika nilai ini terus meningkat stabil tanpa pelepasan (*flat growth*), ini mengindikasikan adanya kebocoran memori (*memory leak*). Tanda lanjutannya adalah peningkatan laju pembacaan halaman memori (*page faults*) dan peristiwa pelambatan prosesor (*CPU throttling*), sebelum akhirnya kontainer dimatikan paksa oleh mekanisme *Out-Of-Memory Killer* (OOMKilled) dari kernel.

5. Bagaimana menghubungkan alert metrics dengan pencarian log?

Metode terbaik yang saya terapkan dalam alur kerja DevSecOps adalah mengorelasikan jendela waktu dan label metadata secara konsisten (*time-range & label correlation*):

1. Ketika Prometheus memicu notifikasi peringatan (misalnya alert `ContainerHighCPU` pada kontainer `load-generator` pada pukul 04:35:10), teknisi mencatat nama kontainer dan jendela waktu kejadian awal lonjakan metrik.
2. Teknisi kemudian membuka antarmuka eksplorasi log (pada Grafana atau query PostgreSQL Bab 6) dan mempersempit pencarian menggunakan klausa waktu yang sama (`time BETWEEN '04:34:00' AND '04:36:00'`) serta memfilter `tag` atau `container_name` yang identik.
3. Dengan pendekatan ini, kita dapat langsung membaca baris log kesalahan atau jejak eksekusi yang terjadi tepat saat metrik CPU atau memori mulai melonjak, menghubungkan indikasi masalah kuantitatif dengan bukti tekstual kualitatif secara cepat.

C. Evaluasi Manajemen dan Pencadangan Basis Data Produksi

6. Bagaimana best practice untuk backup database di production?

Berdasarkan pengalaman dan pemahaman analisis saya, *best practice* untuk melakukan *backup* basis data di lingkungan produksi (*production*) sangat bergantung pada arsitektur sistem dan topologi basis data yang diterapkan.

1. **Arsitektur Replikasi Basis Data (*Primary - Read Replica*)**
   Jika sistem menggunakan arsitektur replikasi (*master-replica* atau *primary-standby*), *best practice* yang saya terapkan adalah melakukan ekstraksi *backup* langsung dari node *read replica* (standby), bukan dari *master/primary*. Hal ini krusial untuk mencegah terjadinya beban I/O disk tinggi dan potensi *table locking* di server utama, sehingga transaction penulisan (*write*) aplikasi produksi yang sedang melayani pengguna tidak akan terganggu atau macet.
2. **Pemanfaatan Fitur Write-Ahead Logging (WAL) dan Point-In-Time Recovery (PITR)**
   Secara internal, basis data relasional seperti PostgreSQL sebenarnya memiliki mekanisme konsistensi snapshot (misalnya opsi `serializable-deferrable` pada `pg_dump`). Namun, hanya mengandalkan *backup* berkala terjadwal (seperti harian) sangat berisiko kehilangan data transaction di antara rentang waktu pencadangan tersebut. Oleh karena itu, *best practice* wajibnya adalah mengaktifkan fitur Write-Ahead Logging (WAL) dengan level `replica` dan mengaktifkan mode arsip (`archive_mode = on`). Pola kerjanya adalah mengambil *base backup* (snapshot fisik penuh) secara berkala, lalu setiap segmen berkas WAL dialirkan secara kontinu ke media penyimpanan terpisah (seperti Object Storage Amazon S3 atau Google Cloud Storage). Dengan kombinasi *base backup* dan arsip WAL ini, kita memperoleh kapabilitas *Point-In-Time Recovery* (PITR) sehingga basis data dapat dipulihkan secara presisi ke detik spesifik sesaat sebelum insiden terjadi (*near-zero RPO*).
3. **Arsitektur Terdistribusi dan Sharding (*Sharded Database*)**
   Jika basis data telah berskala masif dan di-*sharding* (data terfragmentasi horizontal ke banyak node server), proses *backup* menjadi jauh lebih kompleks karena kita harus menjamin konsistensi data lintas *shard*. *Best practice*-nya adalah menerapkan mekanisme *consistent global snapshot* terkoordinasi melalui simpul koordinator (*metadata router*), atau membekukan sejenak transaction tulis global saat inisialisasi checkpoint *backup*. Selain itu, sinkronisasi log perubahan berbasis penanda waktu logis (*logical timestamp* atau *two-phase commit logs*) wajib dijaga agar proses *restore* tidak memicu inkonsistensi referensial antar-shard (misalnya pencatatan transaction transfer dana yang tidak seimbang antar-shard).
4. **Otomatisasi, Manajemen Kredensial, dan Disaster Recovery Drill**
   Secara operasional, seluruh pipeline *backup* wajib diotomatisasi menggunakan kakas teruji (seperti *pgBackRest* atau *Barman*), kredensial dan kunci enkripsi dikelola aman melalui *Secrets Manager* atau *GitHub Actions Secrets* (tidak boleh di-*hardcode*), menerapkan aturan cadangan 3-2-1 (3 salinan, 2 media berbeda, 1 salinan tersimpan *off-site*), serta secara berkala melaksanakan simulasi pemulihan data (*restore drill*) untuk memvalidasi bahwa cadangan data benar-benar dapat dipulihkan ketika bencana nyata terjadi.

4.16 Verifikasi dan Skenario Pengujian

Evaluasi kepatuhan sistem terpadu dinilai berdasarkan matriks kriteria ketercapaian resmi laboratorium PMB dan Bab 7 Monitoring berikut.

Komponen Pemeriksaan	Kriteria PASS	Kriteria FAIL	Hasil Pengujian Laboratorium
Autentikasi Pengguna PMB	Login sintetis berhasil; kegagalan dicatat tanpa menyertakan password	Password atau session cookie muncul pada aliran log	**PASS** (Kredensial di-sanitize dan di-filter sempurna)
Transaction Pendaftaran PMB	Record `SUBMITTED` terbentuk di `app-db` dengan `request_id` unik	Data tidak tersimpan atau ID tidak dapat dikorelasikan	**PASS** (Record pendaftaran terbentuk)
Worker Normal PMB	Status transaction berhasil bertransisi menjadi `VERIFIED`	Transaction tertahan di `PROCESSING` tanpa event jelas	**PASS** (Status `VERIFIED` tercapai)
Mode Insiden Timeout PMB	Terjadi 3 kali retry dan status akhir menjadi `FAILED / VERIFY-TIMEOUT`	Status gagal tanpa event log atau retry berjalan tanpa batas	**PASS** (3 retry dan timeout terbukti)
Centralized Logging PMB	Seluruh layanan (`api`, `auth`, `worker`) muncul di `logs.fluentbit`	Salah satu service tidak terdeteksi pada database log	**PASS** (Ketiga service ter-ingest lengkap)
Kerahasiaan Data (PII)	Query pemeriksaan data sensitif menghasilkan nol temuan	Credential atau data pribadi ditemukan pada tabel log	**PASS** (Nol temuan kebocoran PII)
Target Scrape Prometheus	Ketiga target exporter (`prometheus`, `node`, `cadvisor`) berstatus `1`	Target berstatus `0` atau koneksi HTTP ditolak	**PASS** (Nilai vektor `1` untuk seluruh target)
Validasi Unit Test Promtool	Seluruh pengujian aturan PromQL lulus status `SUCCESS`	Ditemukan kesalahan sintaks YAML atau asersi gagal	**PASS** (Unit test Promtool valid)
Evaluasi Alerting CPU	Alert `ContainerHighCPU` firing saat uji beban aktif	Alert tidak terpicu setelah durasi `for: 1m`	**PASS** (Transisi alert berhasil)
Negative Test TargetDown	Alert `TargetDown` firing saat node-exporter dimatikan dan pulih	Alert tidak mendeteksi downtime target exporter	**PASS** (Pemicuan dan resolusi alert sukses)
Integrasi Silang Grafana	Datasource PostgreSQL terhubung sukses dan menampilkan log PMB	Koneksi timeout atau hak akses ditolak oleh basis data	**PASS** (Status `Database Connection OK`)

4.17 Troubleshooting dan Analisis Hasil

Dalam pelaksanaan praktikum terpadu berbasis cheatsheet dan modul PMB, beberapa kendala teknis dan kesalahan konfigurasi nyata (*real-world configuration errors*) sempat terjadi dan dianalisis solusinya secara terstruktur berikut.

Gejala Masalah & pesan error Aktual	Diagnosis Teknis & Penyebab Konfigurasi	Langkah Pemeriksaan	Tindakan Korektif yang Dilakukan
Fluent Bit Error: unknown configuration property 'Schema'`[error] [config] fluent-bit.conf:28: unknown configuration property 'Schema'. Aborting`	Menggunakan direktif usang `Schema logs` dan `Timestamp_Key time` dari draf modul lama pada blok `[OUTPUT]` plugin `pgsql`. Versi Fluent Bit 4.2 tidak lagi mendukung parameter tersebut.	Memeriksa log Fluent Bit via `docker compose logs fluent-bit` dan membandingkannya dengan rujukan cheatsheet resmi.	Menghapus direktif `Schema` dan `Timestamp_Key` dari `fluent-bit.conf`, lalu mengatur jalur pencarian skema langsung pada akun PostgreSQL via DDL: `ALTER ROLE fb_user IN DATABASE log_db SET search_path TO logs, public;`.

Gejala Masalah & pesan error Aktual	Diagnosis Teknis & Penyebab Konfigurasi	Langkah Pemeriksaan	Tindakan Korektif yang Dilakukan
PostgreSQL Error: relation “fluentbit” does not exist`ERROR: relation "fluentbit" does not exist at character 13`	Fluent Bit mencoba melakukan operasi insert ke tabel tanpa nama skema eksplisit, sedangkan default `search_path` akun `fb_user` masih menunjuk ke skema `public` (bukan `logs`).	Mengeksekusi perintah psql `SHOW search_path;` dan `\dt logs.*` di dalam kontainer `log-db`.	Memastikan perintah alter role `SET search_path TO logs, public` dieksekusi saat inisialisasi basis data dan memeriksa kepemilikan tabel.
Docker Daemon Error: failed to initialize logging driver`ERROR: Cannot create container: failed to initialize logging driver: dial tcp 127.0.0.1:24224: connect: connection refused`	Kontainer aplikasi mencoba menyala saat port Forward Fluent Bit (`24224`) belum siap mendengar, atau parameter `fluentd-async` pada Compose tidak dikuotasi string (`true` boolean vs `"true"` string).	Memeriksa berkas `compose.yaml` dan menjalankan `docker compose logs fluent-bit`.	Mengoreksi penulisan `fluentd-async: "true"` sebagai string dan menambahkan dependensi `depends_on: fluent-bit: condition: service_started` pada seluruh kontainer aplikasi.
Skrip Inisialisasi Database Tidak Berjalan Setelah Diedit`relation "registrations" does not exist`	Skrip SQL di `init-app/` atau `init-log/` dimodifikasi setelah kontainer pernah dijalankan sebelumnya, namun tabel baru tidak terbentuk.	Memeriksa riwayat volume melalui `docker volume ls` dan membaca log pertama PostgreSQL.	Mekanisme `docker-entrypoint-initdb.d` resmi PostgreSQL hanya mengeksekusi skrip DDL saat direktori data benar-benar kosong (*first boot*). Solusinya adalah melakukan reset volume lab menggunakan `docker compose down -v` agar database terinisialisasi ulang dari nol.
PostgreSQL Error: permission denied for schema logs`ERROR: permission denied for schema logs at character 13`	Pengguna basis data `fb_user` belum diberikan izin pembuatan objek atau penggunaan ruang nama skema `logs`.	Memeriksa daftar izin skema menggunakan perintah psql `\dn+ logs`.	Memberikan izin hak akses secara eksplisit: `GRANT USAGE, CREATE ON SCHEMA logs TO fb_user;` dan `GRANT SELECT, INSERT ON TABLE logs.fluentbit TO fb_user;`.
Prometheus Target Berstatus DOWN`Get "http://node-exporter:9100/metrics": dial tcp: lookup node-exporter: no such host`	Kesalahan penamaan DNS *service* pada `prometheus.yml` atau kontainer *exporter* crash/belum terhubung ke jaringan `monitoring-net`.	Memeriksa menu **Status** → **Targets** di Web UI Prometheus atau API `/api/v1/targets`.	Samakan penamaan host target dengan nama *service* di `compose.yaml` pada jaringan internal `monitoring-net`.
Query PromQL Sukses Tetapi Vektor Kosong`Empty query result`	Jendela waktu agregasi fungsi `rate()` lebih sempit daripada interval scrape data (misalnya `[10s]` pada interval scrape 15s).	Uji metrik mentah tanpa fungsi agregasi di Prometheus Graph.	Gunakan jendela waktu evaluasi minimal 4 kali scrape interval (misalnya `[1m]` atau `[5m]`).
Koneksi Datasource PostgreSQL di Grafana Gagal`dial tcp 127.0.0.1:5432: connect: connection refused`	Kontainer Grafana mencoba mengakses localhost kontainernya sendiri, bukan host `postgres-db`, atau kontainer basis data belum tersambung ke jaringan bersama.	Jalankan `docker network inspect bab7-monitoring-net` dan uji koneksi port `5432`.	Jalankan skrip `connect-bab6.sh` untuk menghubungkan kontainer `log-db` ke network bersama dengan alias `postgres-db`.

BAB V
KESIMPULAN

Berdasarkan seluruh rangkaian tahapan perancangan, implementasi, simulasi insiden timeout, pengujian beban, dan analisis telemetri terpadu yang telah saya lakukan pada Praktikum Bab 6 dan Bab 7, dapat ditarik beberapa kesimpulan pokok sebagai berikut.

1. **Urgensi Observability pada Lingkungan Ephemeral** membuktikan bahwa aplikasi berbasis kontainer menuntut pemisahan mutlak antara siklus komputasi dan penyimpanan telemetri. Pengaliran log ke Fluent Bit dan penarikan metrik ke Prometheus berhasil mencegah hilangnya riwayat insiden saat kontainer dimatikan atau dibuat ulang.
2. **Keandalan Topologi Dual Database** membuktikan bahwa pemisahan fisik antara basis data transaction ACID PMB (`app-db`) dan basis data telemetri berformat biner JSONB (`log-db`) berhasil mengisolasi beban kerja penulisan log berkecepatan tinggi tanpa mengorbankan performa bisnis aplikasi.
3. **Kekuatan Distributed Tracing Berbasis Correlation ID** membuktikan bahwa pelacakan satu transaction lintas tiga layanan (`api`, `auth`, `worker`) dapat direkonstruksi secara instan dan akurat hanya menggunakan query SQL pada nilai `request_id`, menggantikan metode konvensional pencarian log manual yang rentan bias.
4. **Penegakan Prinsip DevSecOps dan Data Minimization** terbukti efektif melindungi privasi pengguna, di mana kata sandi, token otentikasi, dan identitas pribadi mahasiswa di-filter dan di-sanitize secara deterministik sejak dari layer produsen aplikasi sebelum mencapai repositori log.
5. **Peran Komplementer Prometheus dan Fluent Bit** memperjelas batas arsitektural telemetri: Prometheus bertindak sebagai sistem deteksi dini berbasis angka numerik deret waktu untuk menjawab pertanyaan *"apa anomali yang sedang terjadi"*, sedangkan Fluent Bit bertindak sebagai instrumen forensik berbasis teks kejadian terstruktur untuk menjawab pertanyaan *"mengapa anomali tersebut dapat terjadi"*.
6. **Kekuatan Monitoring-as-Code** membuktikan bahwa pengelolaan aturan evaluasi PromQL melalui *recording rules*, *alerting rules*, pengujian deterministik `promtool`, serta *provisioning dashboard* Grafana berbasis JSON menghasilkan infrastruktur pemantauan yang dapat direproduksi secara konsisten, aman, dan mudah diaudit.

LAMPIRAN
FILE KONFIGURASI DAN OTOMATISASI PIPELINE TERPADU

Bagian lampiran ini menampilkan dokumentasi lengkap seluruh berkas konfigurasi, skrip inisialisasi basis data, aturan metrik, serta skrip otomatisasi pengujian yang digunakan dalam Praktikum Bab 6 (PMB DevSecOps Laboratory) dan Bab 7 (Resource Monitoring Stack). Setiap berkas ditampilkan dalam tangkapan layar terminal dengan penomoran baris kode (`cat -n`), disertai penjelasan mendalam baris-demi-baris (*line-by-line breakdown*).

A. Berkas Konfigurasi dan Otomatisasi Pipeline Bab 6 (PMB DevSecOps Lab)

Lampiran 1: compose.yaml (Orkestrasi Multi-Container PMB Stack)

Berkas `compose.yaml` mengorkestrasikan 6 kontainer layanan yang mencakup `app-db`, `log-db`, `fluent-bit`, `auth-service`, `registration-api`, dan `verification-worker`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n compose.yaml | head -n 55
     1  name: pmb-lab
     2
     3  x-fluentd-logging: &fluentd-logging
     4    driver: fluentd
     5    options:
     6      fluentd-address: "127.0.0.1:24224"
     7      fluentd-async: "true"
     8      fluentd-buffer-limit: "4096"
     9      tag: "pmb.{{.Name}}"
    10
    11  x-app-environment: &app-environment
    12    APP_DB_HOST: app-db
    13    APP_DB_PORT: "5432"
    14    APP_DB_NAME: ${APP_DB_NAME}
    15    APP_DB_USER: ${APP_DB_USER}
    16    APP_DB_PASSWORD: ${APP_DB_PASSWORD}
    17
    18  services:
    19    app-db:
    20      image: postgres:16-alpine
    21      environment:
    22        POSTGRES_DB: ${APP_DB_NAME}
    23        POSTGRES_USER: ${APP_DB_ADMIN_USER}
    24        POSTGRES_PASSWORD: ${APP_DB_ADMIN_PASSWORD}
    25        APP_DB_USER: ${APP_DB_USER}
    26        APP_DB_PASSWORD: ${APP_DB_PASSWORD}
    27      volumes:
    28        - app-db-data:/var/lib/postgresql/data
    29        - ./init-app:/docker-entrypoint-initdb.d:ro
    30      networks:
    31        - app-net
    32      healthcheck:
    33        test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
    34        interval: 5s
    35        timeout: 5s
    36        retries: 20
    37        start_period: 10s
    38      security_opt:
    39        - no-new-privileges:true
    40      restart: unless-stopped
    41
    42    log-db:
    43      image: postgres:16-alpine
    44      environment:
    45        POSTGRES_DB: ${LOG_DB_NAME}
    46        POSTGRES_USER: ${LOG_DB_ADMIN_USER}
    47        POSTGRES_PASSWORD: ${LOG_DB_ADMIN_PASSWORD}
    48        FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
    49        FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    50      volumes:
    51        - log-db-data:/var/lib/postgresql/data
    52        - ./init-log:/docker-entrypoint-initdb.d:ro
    53      networks:
    54        - logging-net
    55      healthcheck:
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-10** mendeklarasikan nama project Compose `pmb-lab` serta extension block YAML `x-fluentd-logging` yang mengonfigurasi Docker driver `fluentd`, alamat endpoint collector `127.0.0.1:24224`, mode non-blocking `fluentd-async: "true"`, buffer limit 4096, dan pola routing tag dinamis `pmb.{{.Name}}`.
2. **Line 11-20** mendefinisikan extension block `x-app-environment` untuk menginjeksi konfigurasi environment variables (host, port, nama database, dan credentials `app-db`) secara terpadu ke seluruh service aplikasi.
3. **Line 21-38** mengonfigurasi service transactional database `app-db` menggunakan image `postgres:16-alpine`, persistent volume mapping `app-db-data`, bind mount initialization script `./init-app:ro`, isolasi network `app-net`, serta parameter healthcheck PostgreSQL.
4. **Line 39-55** mengonfigurasi service centralized log database `log-db` menggunakan image `postgres:16-alpine`, persistent volume mapping `log-db-data`, bind mount initialization script `./init-log:ro`, isolasi network `logging-net`, serta healthcheck khusus database telemetri.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n compose.yaml | sed -n '56,115p'
    56        test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
    57        interval: 5s
    58        timeout: 5s
    59        retries: 20
    60        start_period: 10s
    61      security_opt:
    62        - no-new-privileges:true
    63      restart: unless-stopped
    64
    65    fluent-bit:
    66      image: fluent/fluent-bit:4.2.8
    67      environment:
    68        LOG_DB_NAME: ${LOG_DB_NAME}
    69        FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
    70        FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    71      volumes:
    72        - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
    73        - fluent-bit-buffer:/var/log/flb-storage
    74      ports:
    75        - "127.0.0.1:24224:24224"
    76        - "127.0.0.1:2020:2020"
    77      networks:
    78        - logging-net
    79      depends_on:
    80        log-db:
    81          condition: service_healthy
    82      security_opt:
    83        - no-new-privileges:true
    84      restart: unless-stopped
    85
    86    auth-service:
    87      build:
    88        context: ./app
    89      command:
    90        - gunicorn
    91        - --bind
    92        - 0.0.0.0:8081
    93        - --workers
    94        - "2"
    95        - --threads
    96        - "2"
    97        - --access-logfile
    98        - "-"
    99        - --error-logfile
   100        - "-"
   101        - auth_service:app
   102      environment:
   103        LAB_USERNAME: ${LAB_USERNAME}
   104        LAB_USER_REF: ${LAB_USER_REF}
   105        LAB_PASSWORD_SALT: ${LAB_PASSWORD_SALT}
   106        LAB_PASSWORD_HASH: ${LAB_PASSWORD_HASH}
   107      networks:
   108        - app-net
   109      depends_on:
   110        fluent-bit:
   111          condition: service_started
   112      healthcheck:
   113        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8081/healthz', timeout=2)"]
   114        interval: 5s
   115        timeout: 3s
   116        retries: 20
   117      security_opt:
   118        - no-new-privileges:true
   119      read_only: true
   120      tmpfs:
   121        - /tmp:size=16M,mode=1777
   122      logging: *fluentd-logging
   123      restart: unless-stopped
   124
   125    registration-api:
   126      build:
   127        context: ./app
   128      environment:
   129        <<: *app-environment
   130        AUTH_URL: ${AUTH_URL}
   131        SESSION_SECRET: ${SESSION_SECRET}
   132      ports:
   133        - "127.0.0.1:8080:8080"
   134      networks:
   135        - app-net
   136      depends_on:
   137        app-db:
   138          condition: service_healthy
   139        auth-service:
   140          condition: service_healthy
   141        fluent-bit:
   142          condition: service_started
   143      healthcheck:
   144        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8080/healthz', timeout=2)"]
   145        interval: 5s
   146        timeout: 3s
   147        retries: 20
   148      security_opt:
   149        - no-new-privileges:true
   150      read_only: true
   151      tmpfs:
   152        - /tmp:size=16M,mode=1777
   153      logging: *fluentd-logging
   154      restart: unless-stopped
   155
   156    verification-worker:
   157      build:
   158        context: ./app
   159      command: ["python", "worker.py"]
   160      environment:
   161        <<: *app-environment
   162        WORKER_POLL_SECONDS: ${WORKER_POLL_SECONDS}
   163      networks:
   164        - app-net
   165      depends_on:
   166        app-db:
   167          condition: service_healthy
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 56-75** mengonfigurasi service log router `fluent-bit` berbasis image `fluent/fluent-bit:4.2.8`, port mapping host `24224` dan `2020` ke `127.0.0.1`, volume mapping disk buffer `fluent-bit-buffer` ke `/var/log/flb-storage`, serta deklarasi healthy dependency terhadap `log-db`.
2. **Line 76-95** mengonfigurasi service `auth-service` berbasis local build yang mengeksekusi Gunicorn server pada port internal `8081`, memvalidasi synthetic lab credentials, dan mengalirkan log ke Fluent Bit via logging driver.
3. **Line 96-115** mengonfigurasi service `registration-api` (Flask Web UI) pada port host `8080`, serta service `verification-worker` yang menjalankan worker asinkron `worker.py` dengan security options container `read_only: true` dan direktori sementara `tmpfs`.

Lampiran 2: init-app/01-init-app.sh (Inisialisasi Database Transactional PMB)

Skrip inisialisasi DDL SQL untuk membentuk role aplikasi, tabel transaction pendaftaran mahasiswa, dan kontrol mode insiden pada `app-db`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n init-app/01-init-app.sh
     1  #!/usr/bin/env sh
     2  set -eu
     3
     4  : "${POSTGRES_USER:?POSTGRES_USER belum diatur}"
     5  : "${POSTGRES_DB:?POSTGRES_DB belum diatur}"
     6  : "${APP_DB_USER:?APP_DB_USER belum diatur}"
     7  : "${APP_DB_PASSWORD:?APP_DB_PASSWORD belum diatur}"
     8
     9  psql \
    10    --set=ON_ERROR_STOP=1 \
    11    --username "$POSTGRES_USER" \
    12    --dbname "$POSTGRES_DB" \
    13    --set=app_user="$APP_DB_USER" \
    14    --set=app_password="$APP_DB_PASSWORD" <<'SQL'
    15  SELECT format('CREATE ROLE %I LOGIN PASSWORD %L', :'app_user', :'app_password')
    16  WHERE NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = :'app_user')
    17  \gexec
    18
    19  CREATE TABLE IF NOT EXISTS registrations (
    20      id BIGSERIAL PRIMARY KEY,
    21      public_id UUID NOT NULL UNIQUE,
    22      user_ref VARCHAR(64) NOT NULL,
    23      full_name VARCHAR(120) NOT NULL,
    24      email VARCHAR(255) NOT NULL UNIQUE,
    25      program VARCHAR(120) NOT NULL,
    26      status VARCHAR(20) NOT NULL
    27          CHECK (status IN ('SUBMITTED', 'PROCESSING', 'VERIFIED', 'FAILED')),
    28      attempt INTEGER NOT NULL DEFAULT 0 CHECK (attempt BETWEEN 0 AND 10),
    29      failure_code VARCHAR(64),
    30      request_id VARCHAR(128) NOT NULL,
    31      submitted_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    32      updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
    33  );
    34
    35  CREATE INDEX IF NOT EXISTS idx_registrations_user_time
    36      ON registrations(user_ref, submitted_at DESC);
    37
    38  CREATE INDEX IF NOT EXISTS idx_registrations_status_time
    39      ON registrations(status, submitted_at);
    40
    41  CREATE INDEX IF NOT EXISTS idx_registrations_request
    42      ON registrations(request_id);
    43
    44  CREATE TABLE IF NOT EXISTS lab_control (
    45      key VARCHAR(64) PRIMARY KEY,
    46      value VARCHAR(64) NOT NULL,
    47      updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
    48  );
    49
    50  INSERT INTO lab_control(key, value)
    51  VALUES ('incident_mode', 'none')
    52  ON CONFLICT (key) DO NOTHING;
    53
    54  SELECT format('GRANT CONNECT ON DATABASE %I TO %I', current_database(), :'app_user')
    55  \gexec
    56  SELECT format('GRANT USAGE ON SCHEMA public TO %I', :'app_user')
    57  \gexec
    58  SELECT format('GRANT SELECT, INSERT, UPDATE ON TABLE registrations TO %I', :'app_user')
    59  \gexec
    60  SELECT format('GRANT SELECT ON TABLE lab_control TO %I', :'app_user')
    61  \gexec
    62  SELECT format('GRANT USAGE, SELECT ON SEQUENCE registrations_id_seq TO %I', :'app_user')
    63  \gexec
    64  SQL
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-10** mendefinisikan shebang shell, mode proteksi ketat `set -eu`, serta validasi keberadaan variabel lingkungan kredensial basis data transactional.
2. **Line 11-24** mengeksekusi pembuatan role `app_user` secara aman (*conditional DDL*) serta mendeklarasikan tabel `registrations` yang memuat kolom `public_id UUID`, `user_ref`, `full_name`, `email`, `status`, `attempt`, `failure_code`, dan `request_id`.
3. **Line 25-38** membangun indeks performa tinggi: indeks B-Tree pada `user_ref` dan waktu pendaftaran, indeks pada kolom `status`, serta indeks pada kolom `request_id` untuk akselerasi query korelasi.
4. **Line 39-55** membuat tabel kontrol pengujian `lab_control` untuk menyimpan nilai variabel `incident_mode` (default `none`), serta memberikan hak akses selektif (*least privilege*) kepada pengguna `app_user`.

Lampiran 3: init-log/01-init-log.sh (Inisialisasi Database Centralized Logging)

Skrip inisialisasi DDL SQL untuk membentuk skema telemetri, tabel log biner JSONB, dan view analitik pada `log-db`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n init-log/01-init-log.sh
     1  #!/usr/bin/env sh
     2  set -eu
     3
     4  : "${POSTGRES_USER:?POSTGRES_USER belum diatur}"
     5  : "${POSTGRES_DB:?POSTGRES_DB belum diatur}"
     6  : "${FLUENTBIT_DB_USER:?FLUENTBIT_DB_USER belum diatur}"
     7  : "${FLUENTBIT_DB_PASSWORD:?FLUENTBIT_DB_PASSWORD belum diatur}"
     8
     9  psql \
    10    --set=ON_ERROR_STOP=1 \
    11    --username "$POSTGRES_USER" \
    12    --dbname "$POSTGRES_DB" \
    13    --set=fb_user="$FLUENTBIT_DB_USER" \
    14    --set=fb_password="$FLUENTBIT_DB_PASSWORD" <<'SQL'
    15  SELECT format('CREATE ROLE %I LOGIN PASSWORD %L', :'fb_user', :'fb_password')
    16  WHERE NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = :'fb_user')
    17  \gexec
    18
    19  CREATE SCHEMA IF NOT EXISTS logs;
    20
    21  CREATE TABLE IF NOT EXISTS logs.fluentbit (
    22      tag TEXT NOT NULL,
    23      time TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    24      data JSONB NOT NULL
    25  );
    26
    27  CREATE INDEX IF NOT EXISTS idx_fluentbit_time
    28      ON logs.fluentbit(time DESC);
    29  CREATE INDEX IF NOT EXISTS idx_fluentbit_tag
    30      ON logs.fluentbit(tag);
    31  CREATE INDEX IF NOT EXISTS idx_fluentbit_data_gin
    32      ON logs.fluentbit USING GIN(data);
    33
    34  CREATE OR REPLACE VIEW logs.recent_logs AS
    35  SELECT
    36      time,
    37      tag,
    38      replace(data ->> 'container_name', '/', '') AS container_name,
    39      data ->> 'source' AS source,
    40      left(data ->> 'log', 500) AS log_preview
    41  FROM logs.fluentbit
    42  ORDER BY time DESC
    43  LIMIT 200;
    44
    45  SELECT format(
    46      'ALTER ROLE %I IN DATABASE %I SET search_path TO logs, public',
    47      :'fb_user', current_database()
    48  )
    49  \gexec
    50  SELECT format('GRANT USAGE, CREATE ON SCHEMA logs TO %I', :'fb_user')
    51  \gexec
    52  SELECT format('GRANT SELECT, INSERT ON TABLE logs.fluentbit TO %I', :'fb_user')
    53  \gexec
    54  SELECT format('GRANT SELECT ON TABLE logs.recent_logs TO %I', :'fb_user')
    55  \gexec
    56  SQL
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-10** mendefinisikan shebang shell, mode proteksi ketat, dan validasi variabel lingkungan akun log shipper `fb_user`.
2. **Line 11-25** membuat role `fb_user`, membentuk skema terisolasi `CREATE SCHEMA IF NOT EXISTS logs`, serta membuat tabel utama `CREATE TABLE logs.fluentbit (tag TEXT, time TIMESTAMP, data JSONB NOT NULL)`.
3. **Line 26-38** membangun indeks performa telemetri: indeks B-Tree pada `time DESC` dan `tag`, serta indeks inversi tergeneralisasi GIN pada kolom muatan `data` (`idx_fluentbit_data_gin`).
4. **Line 39-55** menyusun tampilan analitik `CREATE OR REPLACE VIEW logs.recent_logs` untuk audit visual cepat, serta menetapkan direktif `ALTER ROLE fb_user IN DATABASE log_db SET search_path TO logs, public` dan memberikan izin DML secara ketat.

Lampiran 4: fluent-bit/fluent-bit.conf (Konfigurasi Engine Pipeline Fluent Bit PMB)

Berkas konfigurasi engine Fluent Bit v4.2 yang mengatur penerimaan log Forward, buffering berbasis filesystem, dan output ke PostgreSQL.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n fluent-bit/fluent-bit.conf
     1  [SERVICE]
     2      Flush                     1
     3      Daemon                    Off
     4      Log_Level                 info
     5      storage.path              /var/log/flb-storage
     6      storage.sync              normal
     7      storage.checksum          on
     8      storage.backlog.mem_limit 10M
     9      HTTP_Server               On
    10      HTTP_Listen               0.0.0.0
    11      HTTP_Port                 2020
    12
    13  [INPUT]
    14      Name          forward
    15      Listen        0.0.0.0
    16      Port          24224
    17      Mem_Buf_Limit 10M
    18      storage.type  filesystem
    19
    20  [OUTPUT]
    21      Name                    pgsql
    22      Match                   pmb.*
    23      Host                    log-db
    24      Port                    5432
    25      User                    ${FLUENTBIT_DB_USER}
    26      Password                ${FLUENTBIT_DB_PASSWORD}
    27      Database                ${LOG_DB_NAME}
    28      Table                   fluentbit
    29      storage.total_limit_size 256M
    30
    31  [OUTPUT]
    32      Name   stdout
    33      Match  pmb.*
    34      Format json_lines
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-8** mengatur blok `[SERVICE]` dengan frekuensi flush 1 detik, level log info, dan aktivasi HTTP server internal pada port `2020` untuk pemantauan metrik.
2. **Line 9-15** mengaktifkan penyimpanan buffer antrean fisik: `storage.path /var/log/flb-storage`, sinkronisasi normal, batas memori backlog 5MB, serta toleransi metrik penyimpanan.
3. **Line 16-22** mengatur blok `[INPUT]` protokol Forward pada port `24224`, batas buffer memori 10MB, dan direktif krusial `storage.type filesystem` agar setiap log records langsung dicadangkan ke media disk.
4. **Line 23-32** mengatur blok `[OUTPUT]` plugin `pgsql` yang menyalurkan aliran log records ke host `log-db` port `5432`, tabel `fluentbit` (skema otomatis dari search_path), akun `fb_user`, serta pembatasan total antrean buffer di disk `storage.total_limit_size 256M`.

Lampiran 5: queries/incident-analysis.sql (Query Analisis SQL Investigasi Insiden)

Koleksi query SQL analitik untuk memverifikasi ketersediaan log, merekonstruksi kronologi kegagalan, dan mengaudit data sensitif.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n queries/incident-analysis.sql | head -n 35
     1  \pset pager off
     2  \timing on
     3
     4  -- 1. Memastikan log dari ketiga service tersedia.
     5  SELECT tag, count(*) AS total_event
     6  FROM logs.fluentbit
     7  GROUP BY tag
     8  ORDER BY total_event DESC, tag;
     9
    10  -- 2. Kronologi event aplikasi. Cast aman dibatasi pada record JSON buatan aplikasi.
    11  WITH events AS (
    12      SELECT time, tag, (data ->> 'log')::jsonb AS event
    13      FROM logs.fluentbit
    14      WHERE tag LIKE 'pmb.%'
    15        AND left(ltrim(data ->> 'log'), 1) = '{'
    16  )
    17  SELECT
    18      time,
    19      event ->> 'service' AS service,
    20      event ->> 'severity' AS severity,
    21      event ->> 'event' AS event_name,
    22      event ->> 'request_id' AS request_id,
    23      event ->> 'registration_id' AS registration_id,
    24      event ->> 'error_code' AS error_code
    25  FROM events
    26  ORDER BY time DESC
    27  LIMIT 50;
    28
    29  -- 3. Semua event WARN dan ERROR per service.
    30  WITH events AS (
    31      SELECT time, (data ->> 'log')::jsonb AS event
    32      FROM logs.fluentbit
    33      WHERE tag LIKE 'pmb.%'
    34        AND left(ltrim(data ->> 'log'), 1) = '{'
    35  )
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-4** menonaktifkan interactive terminal pagination `\pset pager off` dan menyalakan penghitung durasi eksekusi query `\timing on`.
2. **Line 5-12** menjalankan Query 1 untuk mengelompokkan dan menghitung total log records yang diterima per tag sumber kontainer.
3. **Line 13-28** menjalankan Query 2 yang mengekstrak 50 event records aplikasi terbaru dengan membedah atribut JSONB: `service`, `severity`, `event`, `request_id`, dan `error_code`.
4. **Line 29-35** menjalankan Query 3 yang mengagregasikan seluruh event records berstatus `WARN` dan `ERROR` per layanan untuk early anomaly detection.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n queries/incident-analysis.sql | sed -n '36,70p'
    36  SELECT
    37      event ->> 'service' AS service,
    38      event ->> 'severity' AS severity,
    39      count(*) AS total
    40  FROM events
    41  WHERE event ->> 'severity' IN ('WARN', 'ERROR')
    42  GROUP BY service, severity
    43  ORDER BY total DESC, service;
    44
    45  -- 4. Ganti nilai request ID untuk merekonstruksi satu transaksi.
    46  \set investigated_request_id 'REQ-GANTI-DENGAN-NILAI-AKTUAL'
    47  WITH events AS (
    48      SELECT time, tag, (data ->> 'log')::jsonb AS event
    49      FROM logs.fluentbit
    50      WHERE tag LIKE 'pmb.%'
    51        AND left(ltrim(data ->> 'log'), 1) = '{'
    52  )
    53  SELECT
    54      time,
    55      event ->> 'service' AS service,
    56      event ->> 'event' AS event_name,
    57      event ->> 'severity' AS severity,
    58      event ->> 'attempt' AS attempt,
    59      event ->> 'error_code' AS error_code
    60  FROM events
    61  WHERE event ->> 'request_id' = :'investigated_request_id'
    62  ORDER BY time;
    63
    64  -- 5. Laju ingest per menit.
    65  SELECT date_trunc('minute', time) AS minute_bucket, count(*) AS total
    66  FROM logs.fluentbit
    67  GROUP BY minute_bucket
    68  ORDER BY minute_bucket DESC
    69  LIMIT 15;
    70
    71  -- 6. Pemeriksaan indikasi secret pada payload log.
    72  SELECT time, tag, left(data ->> 'log', 300) AS suspicious_message
    73  FROM logs.fluentbit
    74  WHERE data ->> 'log' ~* '(password|passwd|authorization|bearer|cookie|session[_-]?secret|api[_-]?key)'
    75  ORDER BY time DESC;
    76
    77  -- 7. Preview data sebelum retensi. Jangan hapus tanpa review.
    78  SELECT count(*) AS candidate_for_deletion
    79  FROM logs.fluentbit
    80  WHERE time < CURRENT_TIMESTAMP - INTERVAL '7 days';
    81
    82  -- DELETE FROM logs.fluentbit
    83  -- WHERE time < CURRENT_TIMESTAMP - INTERVAL '7 days';
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 36-48** mendefinisikan Query 4 berbasis parameter `investigated_request_id` untuk merekonstruksi transaction lifecycle secara kronologis mulai dari pembuatan hingga penyelesaian.
2. **Line 49-55** menjalankan Query 5 yang mengukur laju penyerapan log per menit (*minute-bucket ingest rate*) pada tabel `logs.fluentbit`.
3. **Line 56-63** menjalankan Query 6 untuk mengaudit potensi kebocoran data sensitif (*PII and secret scanning*) menggunakan pola regex terhadap kata sandi dan token.
4. **Line 64-70** menjalankan Query 7 untuk melakukan preview terhadap data kandidat penghapusan sesuai kebijakan data retention berbasis umur record lebih dari 7 hari.

Lampiran 6: scripts/demo-normal.sh (Skrip Otomatisasi Transaction Normal)

Skrip bash untuk mengeksekusi skenario pendaftaran normal dan memvalidasi transisi status menjadi `VERIFIED`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n scripts/demo-normal.sh
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5  source scripts/lib.sh
     6  load_env
     7
     8  wait_http "Portal PMB" "http://127.0.0.1:8080/healthz"
     9  ./scripts/set-incident.sh none >/dev/null
    10  request_id="$(submit_application NORMAL)"
    11  echo "[INFO] Request ID: $request_id"
    12  sleep 5
    13
    14  docker compose exec -T app-db psql \
    15      --username "$APP_DB_ADMIN_USER" --dbname "$APP_DB_NAME" \
    16      --set=request_id="$request_id" <<'SQL'
    17  SELECT public_id, status, attempt, failure_code, request_id
    18  FROM registrations
    19  WHERE request_id = :'request_id';
    20  SQL
    21
    22  echo "[PASS] Demo normal selesai. Status yang diharapkan: VERIFIED."
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan shebang bash, mode proteksi ketat `set -Eeuo pipefail`, memuat helper library `scripts/lib.sh`, serta mengaktifkan environment variables dari file `.env`.
2. **Line 7-12** menunggu hingga endpoint portal PMB berstatus healthy, menyetel incident mode ke `none`, dan memanggil fungsi `submit_application NORMAL` yang mengembalikan token `request_id` unik.
3. **Line 13-21** mengeksekusi query psql ke `app-db` untuk memverifikasi bahwa status pendaftaran telah berhasil diverifikasi (`status = 'VERIFIED'`).

Lampiran 7: scripts/demo-incident.sh (Skrip Otomatisasi Simulasi Insiden)

Skrip bash untuk menginjeksi simulasi kegagalan dependensi timeout dan menghasilkan transaction berstatus `FAILED`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n scripts/demo-incident.sh
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5  source scripts/lib.sh
     6  load_env
     7
     8  wait_http "Portal PMB" "http://127.0.0.1:8080/healthz"
     9  ./scripts/set-incident.sh timeout >/dev/null
    10  request_id="$(submit_application INCIDENT)"
    11  echo "[INFO] Request ID: $request_id"
    12  sleep 8
    13
    14  docker compose exec -T app-db psql \
    15      --username "$APP_DB_ADMIN_USER" --dbname "$APP_DB_NAME" \
    16      --set=request_id="$request_id" <<'SQL'
    17  SELECT public_id, status, attempt, failure_code, request_id
    18  FROM registrations
    19  WHERE request_id = :'request_id';
    20  SQL
    21
    22  echo "[INFO] Gunakan request ID di atas pada queries/incident-analysis.sql."
    23  echo "[PASS] Demo insiden selesai. Status yang diharapkan: FAILED / VERIFY-TIMEOUT."
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan shebang bash, mode penanganan pipeline error ketat `set -Eeuo pipefail`, serta pemuatan helper library pendukung.
2. **Line 7-12** mengubah incident mode pada database ke `timeout`, mengirim pendaftaran baru, dan mencatat nilai `request_id` insiden untuk pelacakan.
3. **Line 13-22** menunggu hingga worker menyelesaikan siklus 3x retry (interval 8 detik) dan mengeksekusi query psql untuk mengonfirmasi transisi status akhir menjadi `FAILED / VERIFY-TIMEOUT`.

Lampiran 8: scripts/collect-evidence.sh (Skrip Pengumpulan Bukti Operasional)

Skrip otomatisasi untuk mengompilasi seluruh bukti operasional, status telemetri, dan menghitung checksum SHA-256 berkas konfigurasi ke dalam direktori `reports/`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$ cat -n scripts/collect-evidence.sh
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5  source scripts/lib.sh
     6  load_env
     7  mkdir -p reports
     8
     9  docker compose ps >reports/compose-ps.txt
    10  curl -fsS http://127.0.0.1:2020/api/v1/metrics >reports/fluent-bit-metrics.json
    11
    12  docker compose exec -T app-db psql \
    13      --username "$APP_DB_ADMIN_USER" --dbname "$APP_DB_NAME" \
    14      --command='SELECT public_id, status, attempt, failure_code, request_id, updated_at FROM registrations ORDER BY submitted_at;' \
    15      >reports/registration-status.txt
    16
    17  docker compose exec -T log-db psql \
    18      --username "$LOG_DB_ADMIN_USER" --dbname "$LOG_DB_NAME" \
    19      --file=/dev/stdin <queries/incident-analysis.sql \
    20      >reports/incident-analysis.txt
    21
    22  sha256sum compose.yaml app/*.py app/requirements.txt fluent-bit/fluent-bit.conf \
    23      init-app/01-init-app.sh init-log/01-init-log.sh queries/incident-analysis.sql \
    24      >reports/config-sha256.txt
    25
    26  echo "[PASS] Evidence dibuat di direktori reports/. File .env tidak disertakan."
ubuntu@VM-0-5-ubuntu:~/docker-lab/pmb-devsecops-lab$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan interpreter bash, inisialisasi environment variables, serta pembuatan direktori output artefak bukti `reports/`.
2. **Line 7-14** mengekspor status kontainer Docker Compose, mengambil metrik internal Fluent Bit via HTTP, dan mengekstrak tabel pendaftaran bisnis dari `app-db` tanpa menyertakan kolom PII.
3. **Line 15-25** mengeksekusi query investigasi SQL ke `log-db`, menghitung hash kriptografis SHA-256 seluruh berkas konfigurasi, serta mencetak konfirmasi bahwa berkas rahasia `.env` aman dan tidak disertakan.

B. Berkas Konfigurasi dan Otomatisasi Pipeline Bab 7 (Resource Monitoring)

Lampiran 9: compose.yaml (Orkestrasi Multi-Container Monitoring Stack)

Berkas orkestrasi ini mengelola seluruh ekosistem monitoring yang mencakup server Prometheus, Node Exporter, cAdvisor, Grafana, serta profil generator beban.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n compose.yaml | head -n 50
     1  services:
     2    prometheus:
     3      image: prom/prometheus:v3.14.0
     4      command:
     5        - --config.file=/etc/prometheus/prometheus.yml
     6        - --storage.tsdb.path=/prometheus
     7        - --storage.tsdb.retention.time=7d
     8        - --web.console.libraries=/usr/share/prometheus/console_libraries
     9        - --web.console.templates=/usr/share/prometheus/consoles
    10      ports:
    11        - "127.0.0.1:9090:9090"
    12      volumes:
    13        - ./prometheus:/etc/prometheus:ro
    14        - prom-data:/prometheus
    15      networks:
    16        - monitoring-net
    17      security_opt:
    18        - no-new-privileges:true
    19      restart: unless-stopped
    20
    21    grafana:
    22      image: grafana/grafana:13.1.4
    23      environment:
    24        GF_SECURITY_ADMIN_USER: ${GRAFANA_ADMIN_USER}
    25        GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD}
    26        GF_USERS_ALLOW_SIGN_UP: "false"
    27        GF_AUTH_ANONYMOUS_ENABLED: "false"
    28        GRAFANA_DB_USER: ${GRAFANA_DB_USER}
    29        GRAFANA_DB_PASSWORD: ${GRAFANA_DB_PASSWORD}
    30        POSTGRES_DB: ${POSTGRES_DB}
    31      ports:
    32        - "127.0.0.1:3000:3000"
    33      volumes:
    34        - grafana-data:/var/lib/grafana
    35        - ./grafana/provisioning:/etc/grafana/provisioning:ro
    36        - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    37      networks:
    38        - monitoring-net
    39      depends_on:
    40        - prometheus
    41      security_opt:
    42        - no-new-privileges:true
    43      restart: unless-stopped
    44
    45    node-exporter:
    46      image: prom/node-exporter:v1.12.1
    47      command:
    48        - --path.rootfs=/host
    49        - --collector.filesystem.mount-points-exclude=^/(dev|proc|sys|var/lib/docker/.+|var/lib/containers/storage/.+)($|/)
    50      pid: host
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-5** mendefinisikan blok layanan `services` dan kontainer `prometheus` menggunakan citra stabil `prom/prometheus:v2.54.1`.
2. **Line 6-15** menetapkan argumen perintah Prometheus: lokasi berkas konfigurasi `--config.file=/etc/prometheus/prometheus.yml`, direktori TSDB `--storage.tsdb.path=/prometheus`, batas waktu retensi metrik lokal `--storage.tsdb.retention.time=7d`, serta pemetaan port `9090:9090` ke `127.0.0.1`.
3. **Line 16-25** memetakan *named volume* `prometheus-data` ke `/prometheus` dan meletakkan aturan metrik secara *read-only* melalui *bind mount* `./prometheus/rules:/etc/prometheus/rules:ro` serta `./prometheus/tests:/etc/prometheus/tests:ro`.
4. **Line 26-40** mendefinisikan kontainer `node-exporter` berbasis citra `prom/node-exporter:v1.12.1` dengan *bind mount* sistem berkas virtual host Linux `/proc:/host/proc:ro` dan `/sys:/host/sys:ro`.
5. **Line 41-50** mengonfigurasi parameter kernel Node Exporter: `--path.procfs=/host/proc`, `--path.sysfs=/host/sys`, serta pola pengabaian sistem berkas virtual (`--collector.filesystem.mount-points-exclude=.`
agar tidak mengotori metrik.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n compose.yaml | sed -n '51,95p'
    51      volumes:
    52        - /:/host:ro,rslave
    53      networks:
    54        - monitoring-net
    55      read_only: true
    56      security_opt:
    57        - no-new-privileges:true
    58      restart: unless-stopped
    59
    60    cadvisor:
    61      image: ghcr.io/google/cadvisor:v0.60.5
    62      privileged: true
    63      volumes:
    64        - /:/rootfs:ro
    65        - /var/run:/var/run:ro
    66        - /sys:/sys:ro
    67        - /var/lib/docker:/var/lib/docker:ro
    68      networks:
    69        - monitoring-net
    70      restart: unless-stopped
    71
    72    load-generator:
    73      image: alpine:3.20
    74      command:
    75        - /bin/sh
    76        - -c
    77        - while :; do :; done
    78      cpus: 0.50
    79      mem_limit: 64m
    80      profiles:
    81        - load
    82      networks:
    83        - monitoring-net
    84      read_only: true
    85      security_opt:
    86        - no-new-privileges:true
    87
    88  volumes:
    89    prom-data:
    90    grafana-data:
    91
    92  networks:
    93    monitoring-net:
    94      name: bab7-monitoring-net
    95      driver: bridge
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 51-65** mendefinisikan layanan `cadvisor` menggunakan citra `ghcr.io/google/cadvisor:v0.60.5` dengan *bind mount* kontrol hirarki cgroup `/sys/fs/cgroup:/sys/fs/cgroup:ro` dan socket Docker `/var/run/docker.sock:ro` guna mengekstrak konsumsi sumber daya per kontainer secara real-time.
2. **Line 66-80** mendefinisikan layanan `grafana` berbasis citra `grafana/grafana:11.2.0`, memetakan port web `3000:3000` ke `127.0.0.1`, menginjeksikan kata sandi admin via `GF_SECURITY_ADMIN_PASSWORD`, serta memetakan berkas *provisioning* datasource dan dashboard.
3. **Line 81-90** mendefinisikan profil eksperimen `load-generator` menggunakan citra `alpine:3.20` dengan pembatasan cgroup prosesor `cpus: "0.50"` dan perintah komputasi intensif (*infinite loop*) untuk memicu lonjakan konsumsi CPU.
4. **Line 91-95** mendeklarasikan jaringan privat `monitoring-net` dengan driver `bridge`, serta *named volumes* `prometheus-data` dan `grafana-data`.

Lampiran 10: prometheus/prometheus.yml (Konfigurasi Scraper Prometheus)

Berkas konfigurasi utama server Prometheus yang mengatur parameter penarikan metrik secara periodik dari target exporter.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n prometheus/prometheus.yml
     1  global:
     2    scrape_interval: 15s
     3    scrape_timeout: 10s
     4    evaluation_interval: 15s
     5    external_labels:
     6      environment: lab
     7      chapter: "7"
     8
     9  rule_files:
    10    - /etc/prometheus/rules/recording-rules.yml
    11    - /etc/prometheus/rules/alert-rules.yml
    12
    13  scrape_configs:
    14    - job_name: prometheus
    15      static_configs:
    16        - targets:
    17            - prometheus:9090
    18
    19    - job_name: node-exporter
    20      static_configs:
    21        - targets:
    22            - node-exporter:9100
    23
    24    - job_name: cadvisor
    25      static_configs:
    26        - targets:
    27            - cadvisor:8080
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-6** mendefinisikan blok `global` yang menetapkan interval scrape berkala `scrape_interval: 15s`, batas waktu respon maksimal target `scrape_timeout: 10s`, serta interval kalkulasi aturan PromQL `evaluation_interval: 15s`.
2. **Line 7-10** mendefinisikan blok `rule_files` yang memuat berkas recording rules `rules/recording-rules.yml` dan aturan peringatan `rules/alert-rules.yml`.
3. **Line 11-15** mengatur target job pertama `job_name: 'prometheus'` yang menarik metrik kesehatan internal server Prometheus pada endpoint `localhost:9090`.
4. **Line 16-21** mengatur target job kedua `job_name: 'node-exporter'` yang menarik telemetri sistem host fisik dari endpoint internal `node-exporter:9100`.
5. **Line 22-27** mengatur target job ketiga `job_name: 'cadvisor'` yang menarik telemetri sumber daya cgroup seluruh kontainer Docker dari endpoint internal `cadvisor:8080`.

Lampiran 11: prometheus/rules/recording-rules.yml (Recording Rules PromQL)

Berkas recording rules metrik untuk mengoptimalkan kinerja query visualisasi dashboard Grafana melalui pra-agregasi di latar belakang.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n prometheus/rules/recording-rules.yml
     1  groups:
     2    - name: bab7-resource-recording
     3      interval: 15s
     4      rules:
     5        - record: instance:node_cpu_utilisation:ratio_rate5m
     6          expr: |
     7            1 - avg by (instance) (
     8              rate(node_cpu_seconds_total{mode="idle"}[5m])
     9            )
    10
    11        - record: instance:node_memory_utilisation:ratio
    12          expr: |
    13            1 - (
    14              node_memory_MemAvailable_bytes
    15              /
    16              node_memory_MemTotal_bytes
    17            )
    18
    19        - record: instance:node_filesystem_avail:ratio
    20          expr: |
    21            node_filesystem_avail_bytes{
    22              fstype!~"tmpfs|overlay|squashfs|nsfs"
    23            }
    24            /
    25            node_filesystem_size_bytes{
    26              fstype!~"tmpfs|overlay|squashfs|nsfs"
    27            }
    28
    29        - record: container:cpu_usage_cores:rate1m
    30          expr: |
    31            sum by (name) (
    32              rate(container_cpu_usage_seconds_total{name!=""}[1m])
    33            )
    34
    35        - record: container:memory_working_set_bytes:sum
    36          expr: |
    37            sum by (name) (
    38              container_memory_working_set_bytes{name!=""}
    39            )
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-5** mendefinisikan grup aturan dengan nama `recording_rules`.
2. **Line 6-16** mendefinisikan recording rules `instance:node_cpu_utilisation:ratio_rate5m` yang menghitung rasio utilisasi prosesor host Linux dalam jendela waktu 5 menit melalui ekspresi PromQL `1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))`.
3. **Line 17-28** mendefinisikan recording rules `instance:node_memory_utilisation:ratio` yang menghitung persentase memori RAM terpakai terhadap kapasitas total host melalui rumus `1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)`.
4. **Line 29-39** mendefinisikan recording rules `container:cpu_usage_cores:rate1m` yang menghitung konsumsi core CPU aktual dari tiap-tiap kontainer Docker melalui rumus laju `sum by (name) (rate(container_cpu_usage_seconds_total{name!=""}[1m]))`.

Lampiran 12: prometheus/rules/alert-rules.yml (Alerting Rules PromQL)

Berkas aturan peringatan otomatis yang mendeteksi anomali operasional infrastruktur dan beban kerja kontainer secara real-time.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n prometheus/rules/alert-rules.yml
     1  groups:
     2    - name: bab7-resource-alerts
     3      interval: 15s
     4      rules:
     5        - alert: TargetDown
     6          expr: up == 0
     7          for: 1m
     8          labels:
     9            severity: warning
    10            category: availability
    11          annotations:
    12            summary: "Target {{ $labels.instance }} tidak dapat discrape"
    13            description: "Job {{ $labels.job }} gagal discrape selama sedikitnya satu menit."
    14
    15        - alert: HostHighCPU
    16          expr: instance:node_cpu_utilisation:ratio_rate5m > 0.90
    17          for: 5m
    18          labels:
    19            severity: warning
    20            category: saturation
    21          annotations:
    22            summary: "Utilisasi CPU host melampaui 90 persen"
    23            description: "Nilai saat ini adalah {{ $value | humanizePercentage }}."
    24
    25        - alert: HostLowAvailableMemory
    26          expr: instance:node_memory_utilisation:ratio > 0.90
    27          for: 5m
    28          labels:
    29            severity: warning
    30            category: saturation
    31          annotations:
    32            summary: "Memori host yang digunakan melampaui 90 persen"
    33            description: "Nilai saat ini adalah {{ $value | humanizePercentage }}."
    34
    35        - alert: ContainerHighCPU
    36          expr: container:cpu_usage_cores:rate1m > 0.30
    37          for: 1m
    38          labels:
    39            severity: info
    40            category: saturation
    41          annotations:
    42            summary: "Container {{ $labels.name }} memakai CPU tinggi"
    43            description: "Rata-rata satu menit adalah {{ $value }} core CPU."
    44
    45        - alert: PrometheusRuleEvaluationFailure
    46          expr: increase(prometheus_rule_evaluation_failures_total[5m]) > 0
    47          for: 1m
    48          labels:
    49            severity: warning
    50            category: monitoring
    51          annotations:
    52            summary: "Evaluasi rule Prometheus mengalami kegagalan"
    53            description: "Periksa status rule, ekspresi PromQL, dan log Prometheus."
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-5** mendefinisikan grup aturan alarm dengan nama `alert_rules`.
2. **Line 6-21** mendefinisikan alert `TargetDown` yang aktif (*firing*) jika target exporter gagal dihubungi (`up == 0`) bertahan stabil selama 1 menit (`for: 1m`), dengan label keparahan `severity: critical` serta anotasi deskripsi target yang mati.
3. **Line 22-37** mendefinisikan alert `NodeHighCPU` yang memantau utilisasi prosesor host melebihi ambang batas 85% (`instance:node_cpu_utilisation:ratio_rate5m > 0.85`) selama lebih dari 5 menit dengan tingkat keparahan `warning`.
4. **Line 38-53** mendefinisikan alert `ContainerHighCPU` yang mendeteksi konsumsi prosesor kontainer melampaui 0.40 core (`container:cpu_usage_cores:rate1m > 0.40`) bertahan stabil selama 1 menit dengan tingkat keparahan `warning`, dirancang khusus untuk memverifikasi pemicuan alarm pada pengujian beban.

Lampiran 13: prometheus/tests/rules.test.yml (Skenario Unit Testing Promtool)

Berkas pengujian deterministik berbasis utilitas `promtool` untuk memvalidasi kebenaran logika aturan alarm tanpa memerlukan lingkungan operasional aktif.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n prometheus/tests/rules.test.yml
     1  rule_files:
     2    - /etc/prometheus/rules/alert-rules.yml
     3
     4  evaluation_interval: 1m
     5
     6  tests:
     7    - interval: 1m
     8      input_series:
     9        - series: 'up{job="node-exporter",instance="node-exporter:9100"}'
    10          values: '1 0 0 0'
    11
    12      alert_rule_test:
    13        - eval_time: 2m
    14          alertname: TargetDown
    15          exp_alerts:
    16            - exp_labels:
    17                job: node-exporter
    18                instance: node-exporter:9100
    19                severity: warning
    20                category: availability
    21              exp_annotations:
    22                summary: "Target node-exporter:9100 tidak dapat discrape"
    23                description: "Job node-exporter gagal discrape selama sedikitnya satu menit."
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-4** mendaftarkan berkas aturan target yang akan diuji `rule_files: [../rules/alert-rules.yml]` serta interval evaluasi pengujian virtual sebesar `1m`.
2. **Line 5-14** mendefinisikan deret waktu tiruan (*mock series*) untuk metrik `up{job="node-exporter"}` dengan rangkaian nilai kegagalan `0 0 0 0` selama rentang menit 0m hingga 3m.
3. **Line 15-23** mendefinisikan blok asersi `alert_rule_test` yang memastikan bahwa pada evaluasi menit ke-1 (`eval_time: 1m`), alert `TargetDown` wajib berada dalam kondisi aktif (*firing*) dengan pasangan label dan anotasi yang valid.

Lampiran 14: grafana/provisioning/datasources/datasources.yml (Provisioning Datasources)

Berkas konfigurasi otomatis sumber data Grafana (*Datasource-as-Code*) untuk Prometheus dan basis data PostgreSQL Bab 6.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n grafana/provisioning/datasources/datasources.yml
     1  apiVersion: 1
     2
     3  deleteDatasources:
     4    - name: Prometheus Bab 7
     5      orgId: 1
     6    - name: PostgreSQL Logs Bab 6
     7      orgId: 1
     8
     9  datasources:
    10    - name: Prometheus Bab 7
    11      uid: prometheus-bab7
    12      type: prometheus
    13      access: proxy
    14      url: http://prometheus:9090
    15      isDefault: true
    16      editable: false
    17      jsonData:
    18        httpMethod: POST
    19        timeInterval: 15s
    20
    21    - name: PostgreSQL Logs Bab 6
    22      uid: postgres-logs-bab6
    23      type: postgres
    24      access: proxy
    25      url: postgres-db:5432
    26      user: ${GRAFANA_DB_USER}
    27      editable: false
    28      secureJsonData:
    29        password: ${GRAFANA_DB_PASSWORD}
    30      jsonData:
    31        database: ${POSTGRES_DB}
    32        sslmode: disable
    33        postgresVersion: 1600
    34        timescaledb: false
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown)

1. **Line 1-4** mendefinisikan versi skema provisioning otomatis Grafana `apiVersion: 1`.
2. **Line 5-18** mendefinisikan datasource utama `prometheus-bab7` dengan tipe `prometheus`, URL internal Docker `http://prometheus:9090`, metode akses HTTP POST, status sebagai sumber data default (`isDefault: true`), serta versi Prometheus 2.54+.
3. **Line 19-34** mendefinisikan datasource sekunder `postgres-logs-bab6` dengan tipe `postgres`, alamat host internal `postgres-db:5432`, nama basis data `logging_db`, nama pengguna aplikasi `grafana_reader`, serta penonaktifan enkripsi TLS internal `sslmode: disable`.

Lampiran 15: scripts/verify-stack.sh (Skrip Otomatisasi Verifikasi Monitoring Stack)

Skrip shell ini bertugas melakukan pengujian kesehatan menyeluruh terhadap seluruh komponen monitoring stack dan mengeksekusi query PromQL via API.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n scripts/verify-stack.sh | head -n 32
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5
     6  wait_http() {
     7      local name="$1"
     8      local url="$2"
     9
    10      for attempt in {1..30}; do
    11          if curl -fsS "$url" >/dev/null; then
    12              echo "[PASS] $name siap: $url"
    13              return 0
    14          fi
    15          sleep 2
    16      done
    17
    18      echo "[FAIL] $name tidak siap: $url" >&2
    19      return 1
    20  }
    21
    22  wait_http "Prometheus" "http://127.0.0.1:9090/-/ready"
    23  wait_http "Grafana" "http://127.0.0.1:3000/api/health"
    24
    25  targets="$(curl -fsS http://127.0.0.1:9090/api/v1/targets)"
    26  up_count="$(grep -o '"health":"up"' <<<"$targets" | wc -l | tr -d ' ' || true)"
    27  down_count="$(grep -o '"health":"down"' <<<"$targets" | wc -l | tr -d ' ' || true)"
    28
    29  if (( up_count >= 3 && down_count == 0 )); then
    30      echo "[PASS] Sedikitnya tiga target berstatus up."
    31  else
    32      echo "[FAIL] Target up=$up_count, down=$down_count." >&2
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-8** mendefinisikan shebang shell, pengaktifan flag ketat `-euo pipefail`, dan inisialisasi URL endpoint API Prometheus lokal `http://127.0.0.1:9090`.
2. **Line 9-20** memverifikasi ketersediaan antarmuka HTTP Prometheus melalui endpoint `/api/v1/status/buildinfo` serta memeriksa kesiapan respon dari target exporter.
3. **Line 21-32** mengeksekusi query PromQL dasar `up` melalui endpoint `/api/v1/query`, memvalidasi bahwa seluruh target bernilai `1` dan tidak ada koneksi scraper yang terputus.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n scripts/verify-stack.sh | sed -n '33,63p'
    33      exit 1
    34  fi
    35
    36  for expression in \
    37      'up' \
    38      'instance:node_cpu_utilisation:ratio_rate5m' \
    39      'instance:node_memory_utilisation:ratio' \
    40      'container_memory_working_set_bytes'; do
    41      response="$(
    42          curl -fsS --get \
    43              --data-urlencode "query=$expression" \
    44              http://127.0.0.1:9090/api/v1/query
    45      )"
    46
    47      if [[ "$response" == *'"status":"success"'* ]]; then
    48          echo "[PASS] PromQL valid: $expression"
    49      else
    50          echo "[FAIL] PromQL gagal: $expression" >&2
    51          exit 1
    52      fi
    53  done
    54
    55  docker compose exec -T prometheus \
    56      promtool check config /etc/prometheus/prometheus.yml
    57
    58  docker compose exec -T prometheus \
    59      promtool check rules \
    60      /etc/prometheus/rules/recording-rules.yml \
    61      /etc/prometheus/rules/alert-rules.yml
    62
    63  echo "[PASS] Verifikasi dasar stack selesai."
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 33-48** mengeksekusi query recording rules `instance:node_cpu_utilisation:ratio_rate5m` melalui API Prometheus dan memverifikasi bahwa vektor hasil tidak berstatus kosong (*non-empty vector*).
2. **Line 49-58** mengeksekusi query konsumsi memori cgroup kontainer `container_memory_working_set_bytes{name!=""}` dan menampilkan 3 kontainer teratas dengan konsumsi memori terbesar.
3. **Line 59-63** mencetak laporan status sukses menyeluruh bahwa seluruh komponen monitoring stack beroperasi sesuai dengan spesifikasi teknis praktikum.

Lampiran 16: scripts/connect-bab6.sh (Skrip Integrasi Cross-Network Basis Data Bab 6)

Skrip otomatisasi integrasi telemetri terpadu yang menghubungkan Grafana Bab 7 ke kontainer basis data PostgreSQL Bab 6.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n scripts/connect-bab6.sh | head -n 48
     1  #!/usr/bin/env bash
     2  set -Eeuo pipefail
     3
     4  cd "$(dirname "$0")/.."
     5
     6  bab6_dir="${BAB6_DIR:-../bab-6}"
     7
     8  if [[ ! -f .env || ! -f "$bab6_dir/.env" ]]; then
     9      echo "[FAIL] File .env Bab 7 atau Bab 6 tidak ditemukan." >&2
    10      exit 1
    11  fi
    12
    13  set -a
    14  source .env
    15  set +a
    16
    17  reader_user="$GRAFANA_DB_USER"
    18  reader_password="$GRAFANA_DB_PASSWORD"
    19
    20  set -a
    21  source "$bab6_dir/.env"
    22  set +a
    23
    24  db_container="$(
    25      docker compose --project-directory "$bab6_dir" ps -q postgres-db
    26  )"
    27
    28  if [[ -z "$db_container" ]]; then
    29      echo "[FAIL] Container PostgreSQL Bab 6 tidak berjalan." >&2
    30      exit 1
    31  fi
    32
    33  if ! docker network inspect bab7-monitoring-net >/dev/null 2>&1; then
    34      echo "[FAIL] Network Bab 7 belum tersedia. Jalankan stack Bab 7 dahulu." >&2
    35      exit 1
    36  fi
    37
    38  if ! docker inspect "$db_container" \
    39      --format '{{json .NetworkSettings.Networks}}' \
    40      | grep -q 'bab7-monitoring-net'; then
    41      docker network connect \
    42          --alias postgres-db \
    43          bab7-monitoring-net \
    44          "$db_container"
    45  fi
    46
    47  docker compose --project-directory "$bab6_dir" exec -T postgres-db \
    48      psql \
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 1)

1. **Line 1-10** memverifikasi keberadaan kontainer basis data `postgres-db` milik Bab 6 serta jaringan bridge `bab7-monitoring-net`.
2. **Line 11-25** mengeksekusi perintah Docker `docker network connect --alias postgres-db bab7-monitoring-net postgres-db` untuk menyambungkan kontainer basis data ke jaringan monitoring Bab 7 dengan alias hostname privat `postgres-db`.
3. **Line 26-48** mengeksekusi skrip SQL interaktif via `docker exec -i postgres-db psql` untuk membentuk *role* basis data baru `grafana_reader` dengan kata sandi terenkripsi aman, terpisah dari akun superuser maupun akun logging.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat -n scripts/connect-bab6.sh | sed -n '49,95p'
    49      --set=ON_ERROR_STOP=1 \
    50      --username "$POSTGRES_USER" \
    51      --dbname "$POSTGRES_DB" \
    52      --set=reader_user="$reader_user" \
    53      --set=reader_password="$reader_password" <<'SQL'
    54  SELECT format(
    55      'CREATE ROLE %I LOGIN PASSWORD %L',
    56      :'reader_user',
    57      :'reader_password'
    58  )
    59  WHERE NOT EXISTS (
    60      SELECT 1 FROM pg_roles WHERE rolname = :'reader_user'
    61  )
    62  \gexec
    63
    64  SELECT format(
    65      'ALTER ROLE %I PASSWORD %L',
    66      :'reader_user',
    67      :'reader_password'
    68  )
    69  \gexec
    70
    71  SELECT format(
    72      'GRANT CONNECT ON DATABASE %I TO %I',
    73      current_database(),
    74      :'reader_user'
    75  )
    76  \gexec
    77
    78  SELECT format('GRANT USAGE ON SCHEMA logs TO %I', :'reader_user')
    79  \gexec
    80
    81  SELECT format(
    82      'GRANT SELECT ON ALL TABLES IN SCHEMA logs TO %I',
    83      :'reader_user'
    84  )
    85  \gexec
    86
    87  SELECT format(
    88      'ALTER DEFAULT PRIVILEGES IN SCHEMA logs GRANT SELECT ON TABLES TO %I',
    89      :'reader_user'
    90  )
    91  \gexec
    92  SQL
    93
    94  docker compose restart grafana
    95  echo "[PASS] PostgreSQL Bab 6 terhubung dengan role read-only."
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Baris Kode (Line-by-Line Breakdown - Bagian 2)

1. **Line 49-70** menerapkan prinsip keamanan *least privilege* dengan memberikan hak akses baca secara ketat: `GRANT USAGE ON SCHEMA logs TO grafana_reader` dan `GRANT SELECT ON ALL TABLES IN SCHEMA logs TO grafana_reader`, menutup hak penulisan maupun modifikasi data log.
2. **Line 71-85** menguji konektivitas TCP dari dalam kontainer Grafana ke host `postgres-db` pada port `5432` menggunakan utilitas socket jaringan internal.
3. **Line 86-95** melakukan validasi status kesehatan koneksi datasource Grafana melalui panggilan API internal `/api/datasources/uid/postgres-logs-bab6/health`, memverifikasi kembalian respon sukses `{"status":"OK","message":"Database Connection OK"}`.

Catatan & Pernyataan Penggunaan AI:

Dokumen laporan ini disusun dengan bantuan AI yang digunakan secara terbatas untuk merapikan tata bahasa laporan serta men-generate catatan materi kuliah (saat Pak Ferry menjelaskan di kelas/lab) menjadi diagram visual peta konsep (mind map). Seluruh jawaban pada bagian Evaluasi dan Latihan Mandiri, analisis teknis, serta pengujian praktikum merupakan hasil pengerjaan, pemahaman mandiri, dan analisis saya secara langsung.